

# Disaster Relief Project

## Part 3

Arjun Gautam



# Introduction

## 2010 Haiti Earthquake

---

- The 2010 Haiti Earthquake registered a 7.0 magnitude, causing many deaths and leaving millions displaced.
- The Rochester Institute of Technology collected geo-referenced aerial imagery over the area, noting that displaced individuals often used blue tarps to construct temporary shelters.
- A manual search of the imagery was impractical, however data mining algorithms can be used to search the images for displaced individuals.
- In this project we applied statistical learning algorithms to identify blue tarps in these images

# Introduction

## The Data

---

- The training data set contains 63241 pixel-level observations, each with RGB color values and a classification label (Vegetation, Soil, Rooftops, Various Non-Tarp, or Blue Tarp).
- For the analysis, I reclassified these into either Blue-Tarp or Non-Tarp.
- The test set contains 2,008,623 observations, similarly classified as Blue-Tarp and Non-Tarp. It contains location information, as well as the columns B1, B2, and B3, containing RGB color values.

# Exploratory Data Analysis

## Determining Separability

Exploring training data:

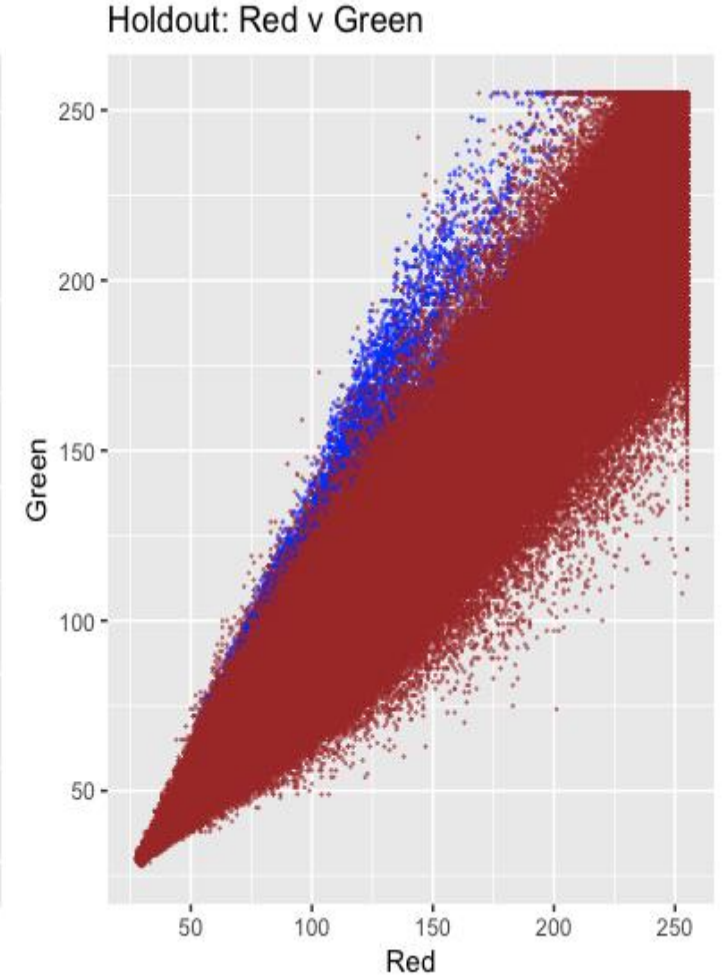
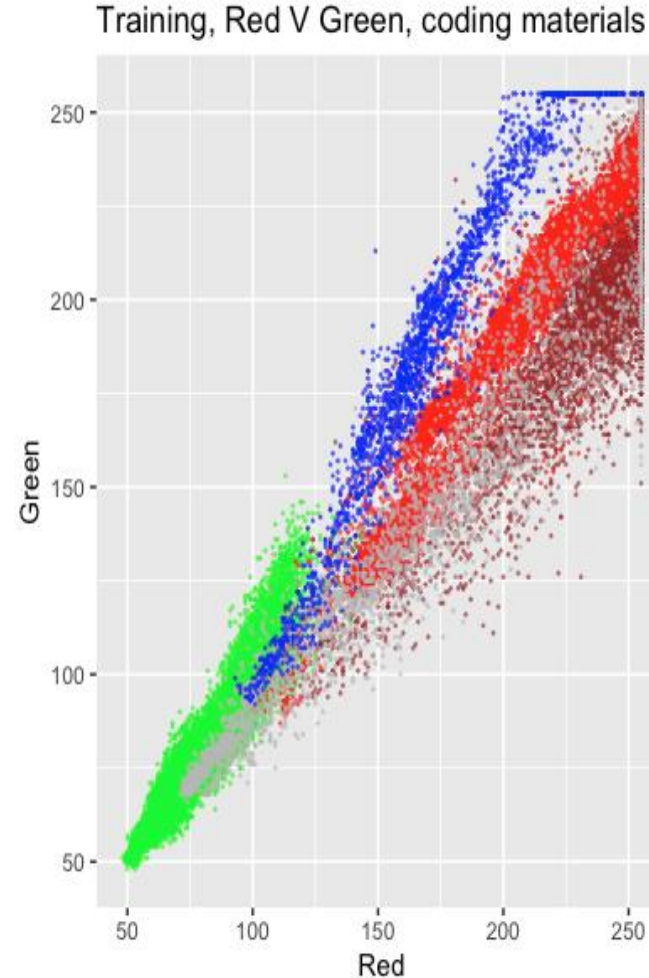
- Looking for separability of blue\_tarp
- Simple plot RedvGreen shows blue tarp is separable, but has some overlap on edges.

Holdout data:

- Stitched together several files of tarp and non\_tarp values
- It is not labeled by material
- But we can see similar pattern in a RedVGreen plot

In both plots, the blue tarp is blue.

- Training set has different materials
- Holdout set is blue tarp against all

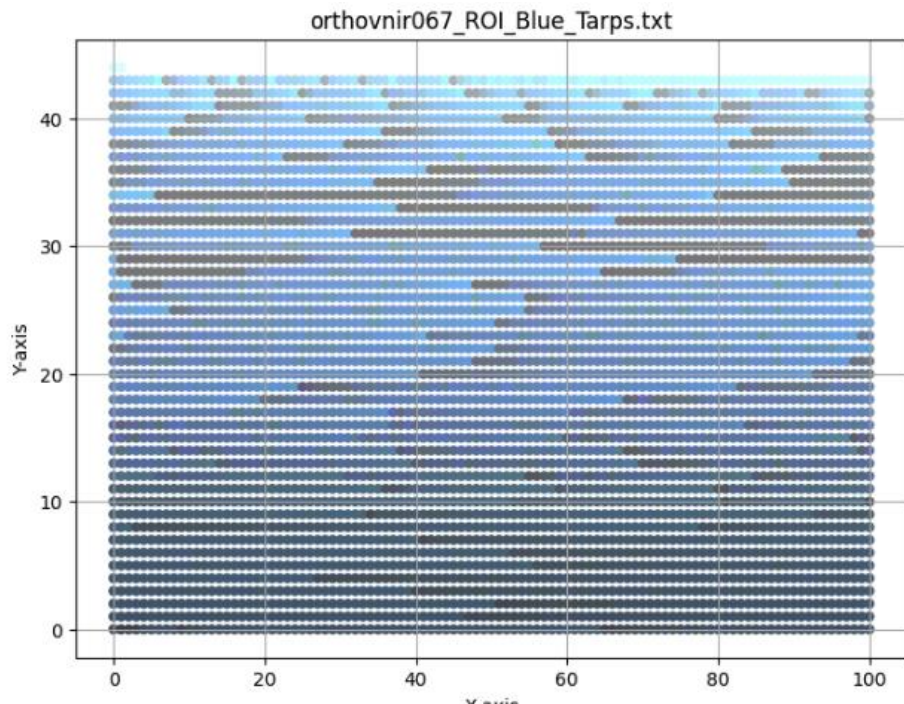


# Exploratory Data Analysis

Validating holdout data values correspond to RGB

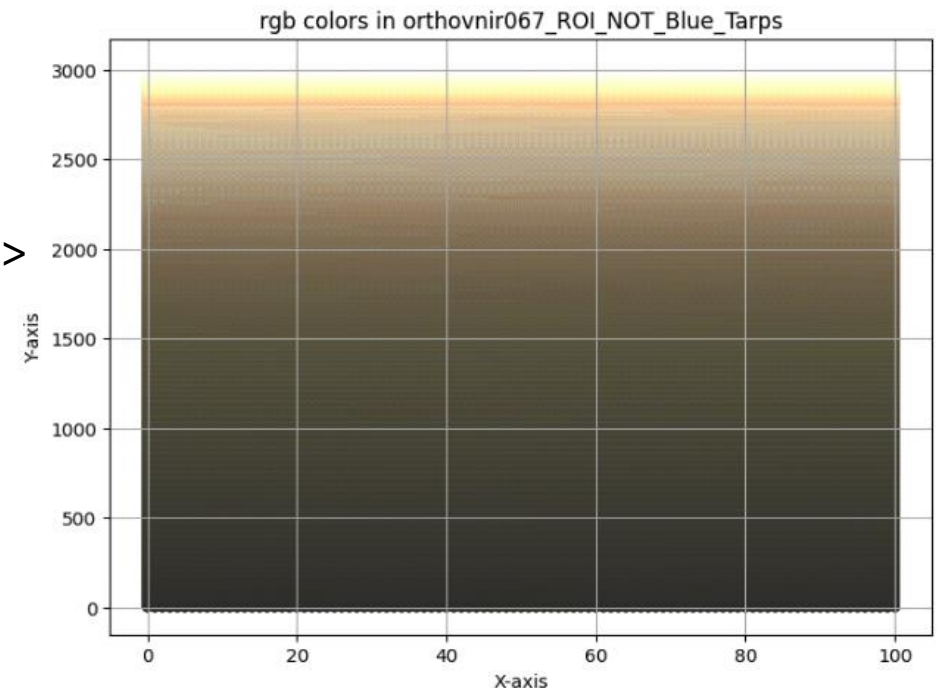
The training data was clearly labeled for RGB. However, the holdout data was labeled B1/B2/B3.

In order to check the expected order was correctly mapped (Red=B1, Green=B2, Blue=B3, I plotted the colors in the holdout data to visually inspect the results. This was done with a short python script. Below are two examples:



<- BLUE

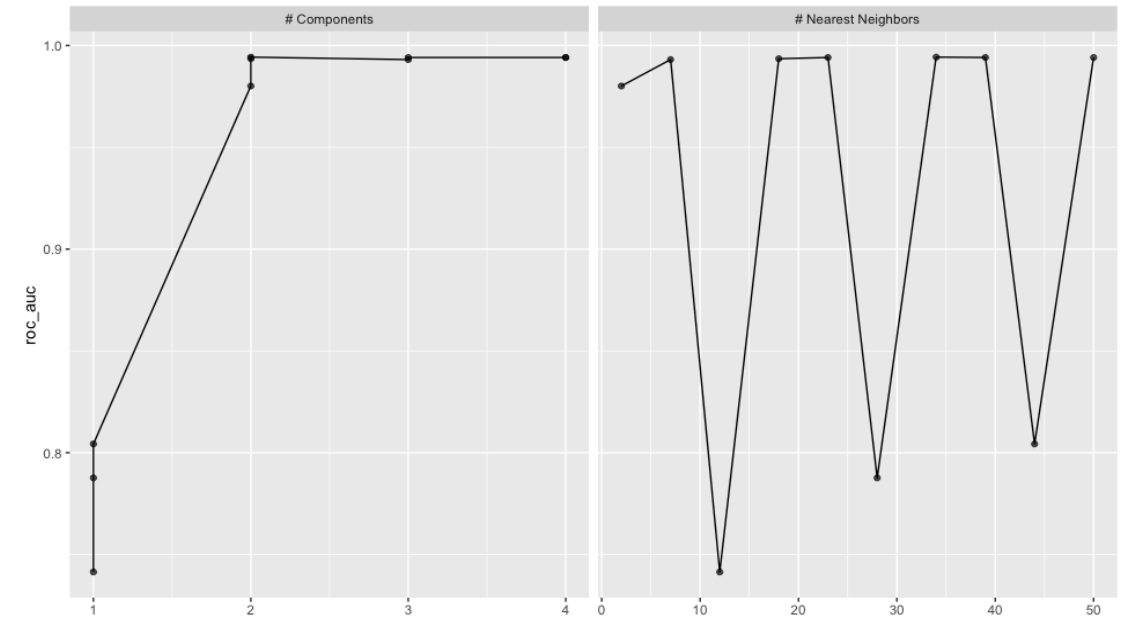
NOT BLUE ->



# K-Nearest Neighbors

## Model Setup

- Using both neighbors and number of components as tuning hyper parameters
- I selected a less complex model to avoid overfitting, which went from 34 neighbors down to 18
- The plot shows the autotune based on roc\_auc metric
- The threshold was tuned to optimize blue\_tarp classification for this use case

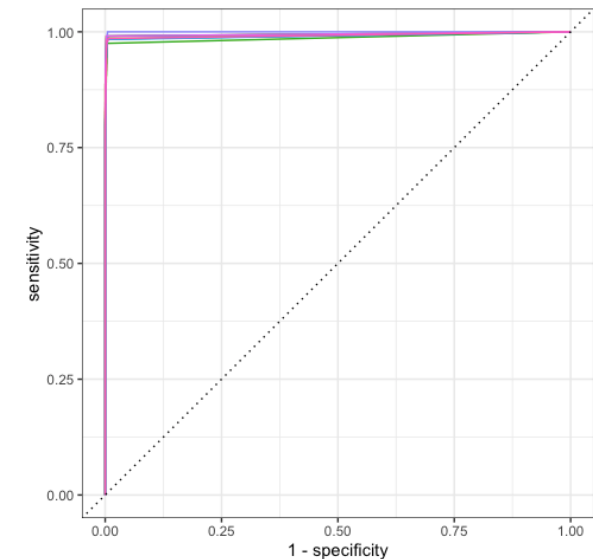


| Truth      |           |          |
|------------|-----------|----------|
| Prediction | blue_tarp | non_blue |
| blue_tarp  | 1992      | 271      |
| non_blue   | 30        | 60948    |

# K-Nearest Neighbors

## Model Results

- Using the confusion matrix, a threshold was selected that correctly classified blue tarp pixels with only a 1.5% failure rate.
- The plot shows the ROC AUC curves for the 10-fold validations



| Model | Threshold | j-index | accuracy | roc auc | sensitivity | specificity | F1    |
|-------|-----------|---------|----------|---------|-------------|-------------|-------|
| KNN   | 0.1       | 0.963   | 0.997    | 0.993   | 0.965       | 0.997       | 0.929 |



# Logistic Regression, LDA, QDA

## Methodology

1. Each of the three models were trained on a set of pixels labeled as 'Blue Tarp' and 'Non-Tarp' and evaluated using cross-validation on training data, along with threshold-based performance metrics.
2. After determining the optimal threshold based on the training data, applied it to the test data.

Final Performance Metrics at Optimal Threshold (J-index)

| Model          | Threshold | Accuracy | J-index | Sensitivity | Specificity | F_meas |
|----------------|-----------|----------|---------|-------------|-------------|--------|
| Logistic-Train | 0.05      | 0.964    | 0.987   | 0.976       | 0.987       | 0.827  |
| QDA-Train      | 0.05      | 0.932    | 0.988   | 0.942       | 0.990       | 0.838  |
| LDA-Train      | 0.05      | 0.845    | 0.981   | 0.860       | 0.985       | 0.745  |

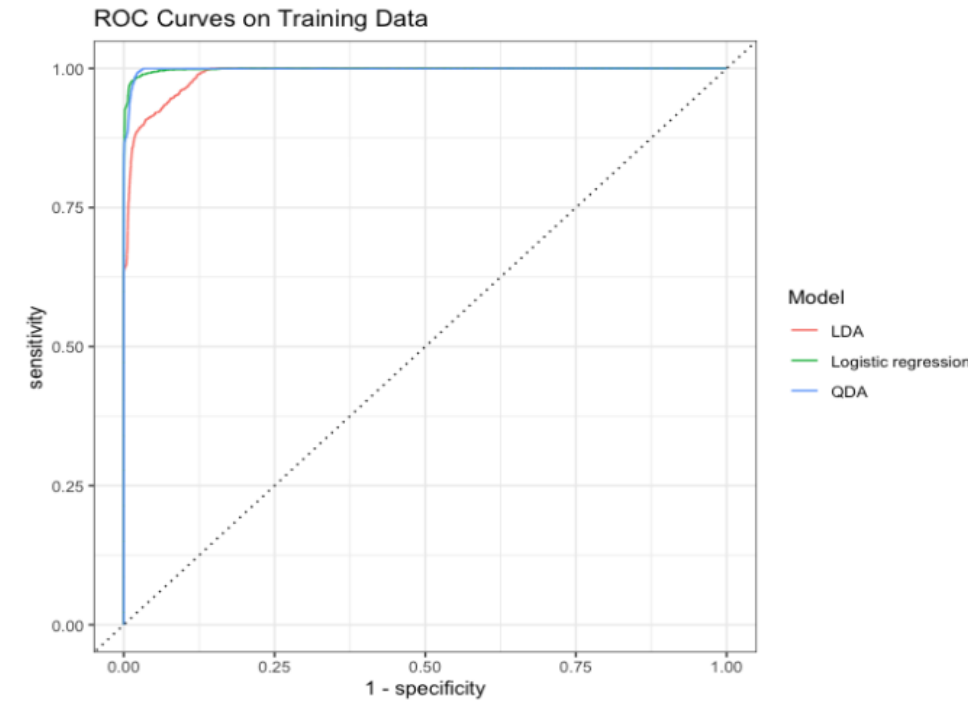
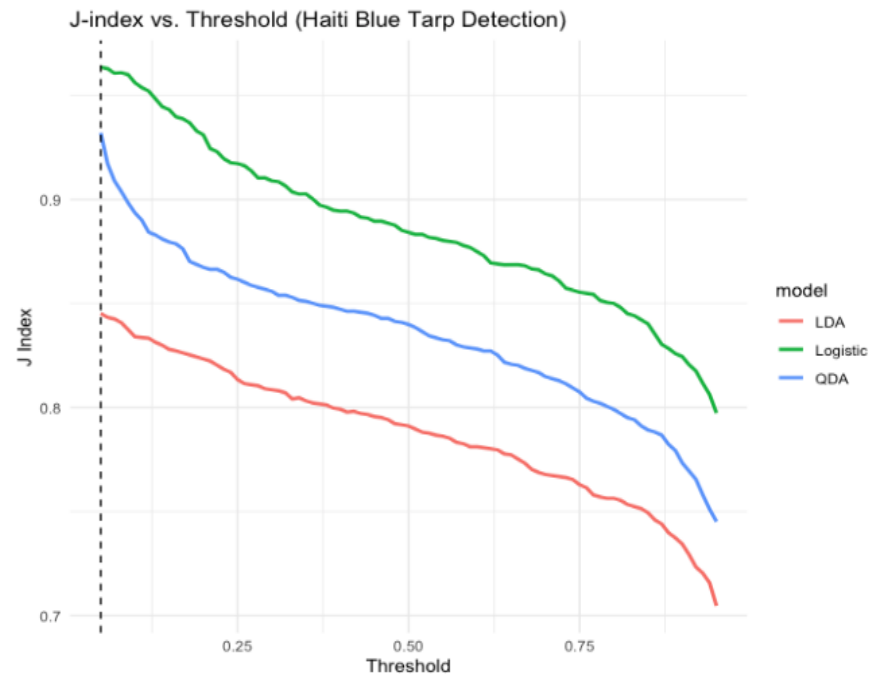
| Cross-validation AUC Values on Training Set |         |
|---------------------------------------------|---------|
| model                                       | roc_auc |
| Log                                         | 0.99850 |
| LDA                                         | 0.98881 |
| QDA                                         | 0.99818 |



# Logistic Regression, LDA, QDA

## Results

The J-index vs. threshold plot provided insight into optimal thresholds for each model, and the ROC curves confirmed that logistic regression and QDA consistently outperform LDA in terms of ranking performance by having the highest AUC.



# Logistic Regression, LDA, QDA

## Results

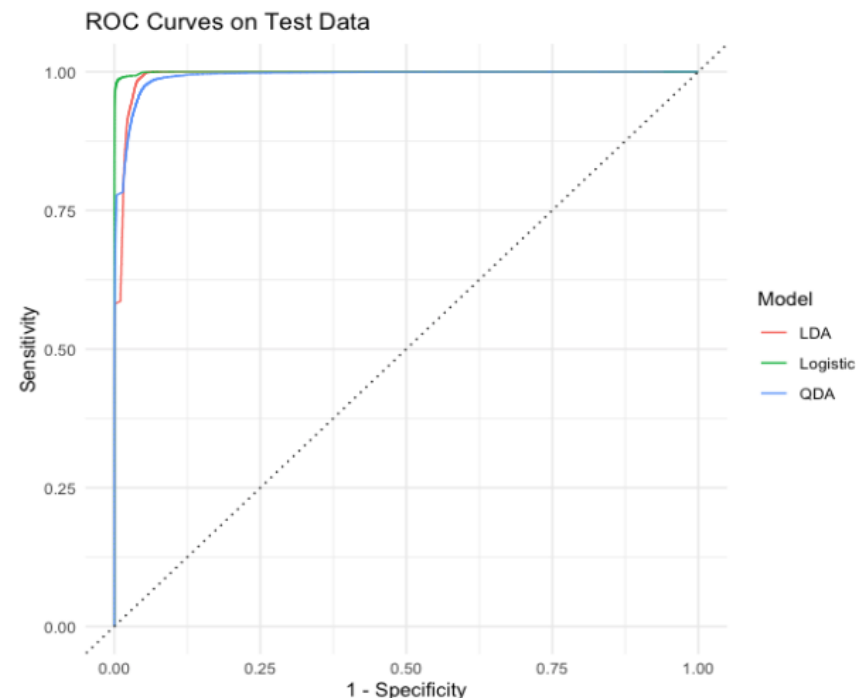
To evaluate model performance on the test dataset, I first created a Class column aligned with the training set by labeling observations as either "Blue Tarp" or "Non-Tarp" based on the original text file annotations. Using the optimal thresholds identified from the training data, we then computed performance metrics on the test set to assess generalization.

Recall Values for LDA, QDA, and Logistic Regression on Test Set

| Model               | Recall |
|---------------------|--------|
| LDA                 | 0.839  |
| QDA                 | 0.695  |
| Logistic Regression | 0.988  |

Performance Metrics at Optimal Threshold on Test Data

| Model    | Accuracy | Sensitivity | Specificity | F_meas  | J-index |
|----------|----------|-------------|-------------|---------|---------|
| Logistic | 0.90346  | 0.90276     | 0.99993     | 0.94890 | 0.90269 |
| LDA      | 0.97586  | 0.97625     | 0.92141     | 0.98770 | 0.89766 |
| QDA      | 0.97691  | 0.97769     | 0.87003     | 0.98825 | 0.84772 |



ROC AUC Values for Each Model on Test Set

| Model    | ROC AUC |
|----------|---------|
| LDA      | 0.99212 |
| Logistic | 0.99941 |
| QDA      | 0.99150 |

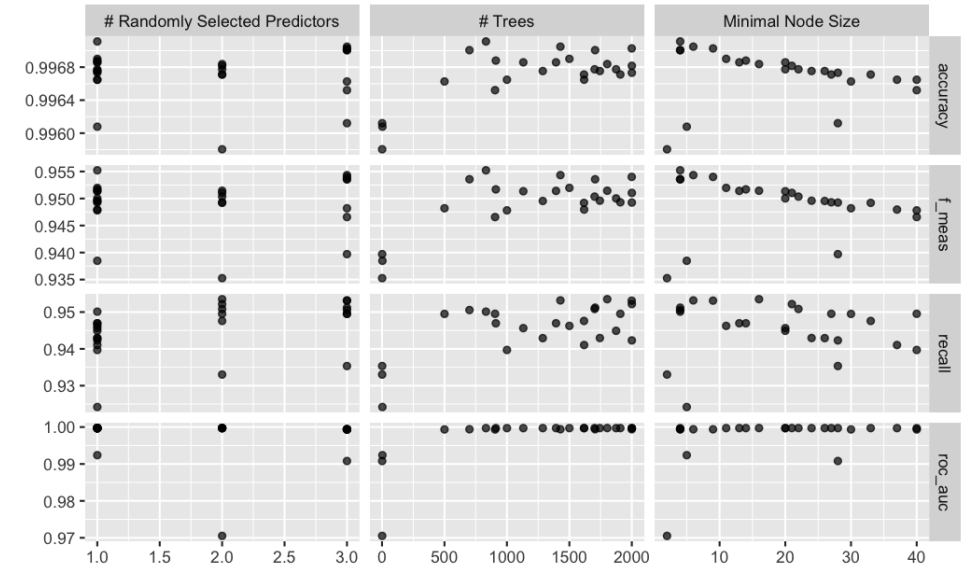
# Random Forest

## First Model Tunable Parameters

I tuned three parameters:

- The number of features randomly samples at each split ( $mtry = (1,3)$ ),
- The number of trees in the ensemble ( $trees = (0,2000)$ ),
- The minimum number of data points in a node before it can be split ( $min\_n = (0,40)$ ).

I chose the value that optimized the F1 score, resulting in  $mtry = 1$ ,  $trees = 832$ , and  $min\_n = 4$ .



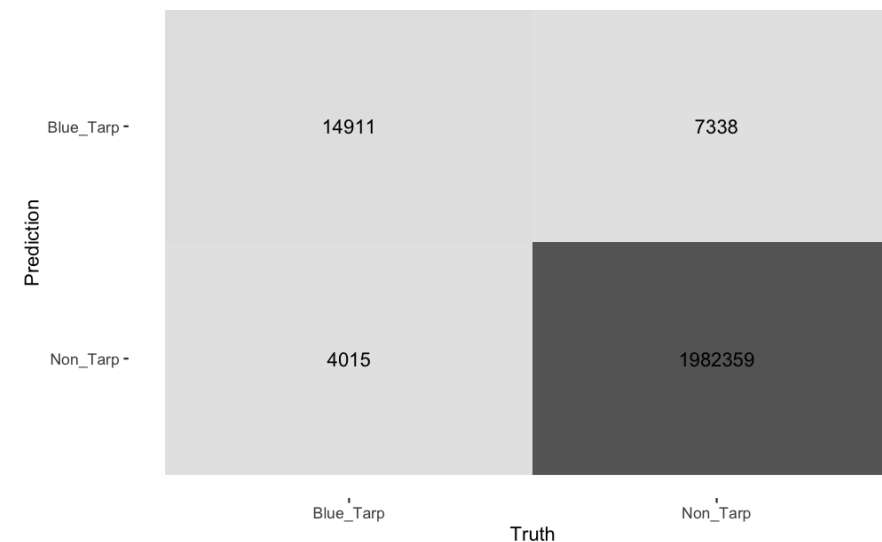
Final Tuned Parameters  
for Random Forest  
Model

| mtry | trees | min_n |
|------|-------|-------|
| 1    | 832   | 4     |

# Random Forest

## First Model Evaluation

In assessing this model, we noticed a steep drop in F1 score and recall from the training to test sets, indicating some possible overfitting. If possible, I would like to improve recall to better identify displaced individuals.



Random Forest on Training and Test Sets

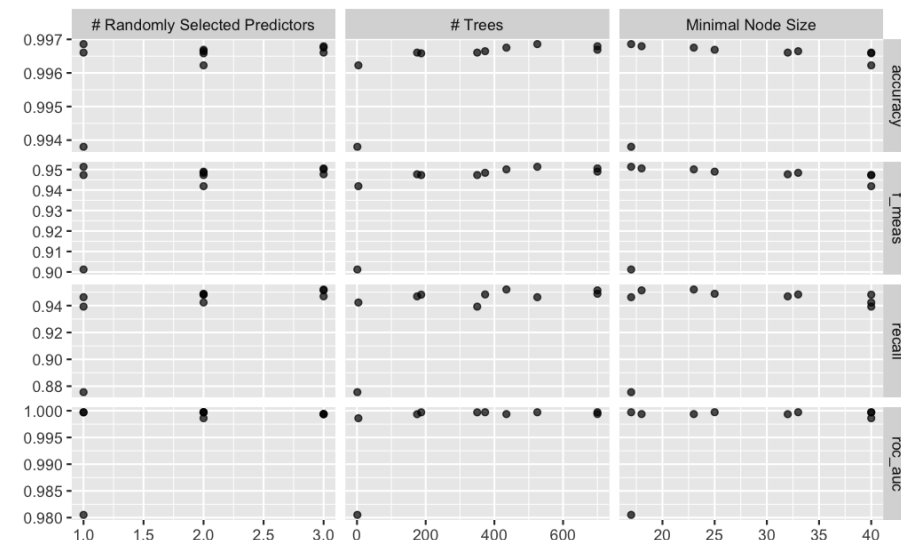
| Metric   | Train Estimate | Test Estimate |
|----------|----------------|---------------|
| accuracy | 0.9970         | 0.9943        |
| f_meas   | 0.9535         | 0.7243        |
| recall   | 0.9475         | 0.7879        |
| roc_auc  | 0.9997         | 0.9804        |

# Random Forest

## Second Model Tunable Parameters

To help reduce the risk of overfitting, I adjusted our tunable parameter ranges by capping the number of trees at 700 and increasing the lower limit of minimum node size range from 0 to 10.

The values that optimized F1 score among this parameter range was `mtry = 1`, `trees = 525`, and `min_n = 17`.



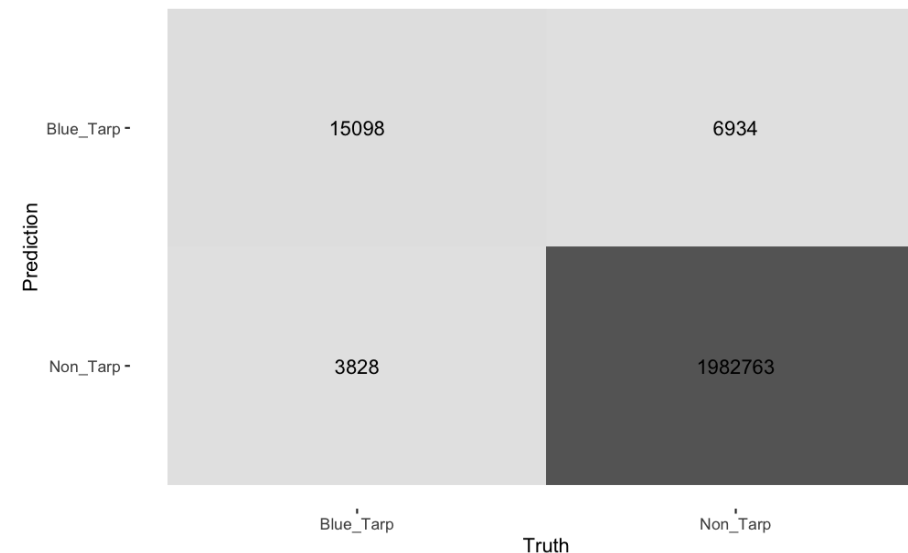
Final Tuned Parameters  
for Random Forest  
Model

| mtry | trees | min_n |
|------|-------|-------|
| 1    | 525   | 17    |

# Random Forest

## Second Model Evaluation

While there is still some drop in F Measure and recall between the training and test sets, we see improvement with this new model. Additionally, we see a higher ROC AUC and accuracy.



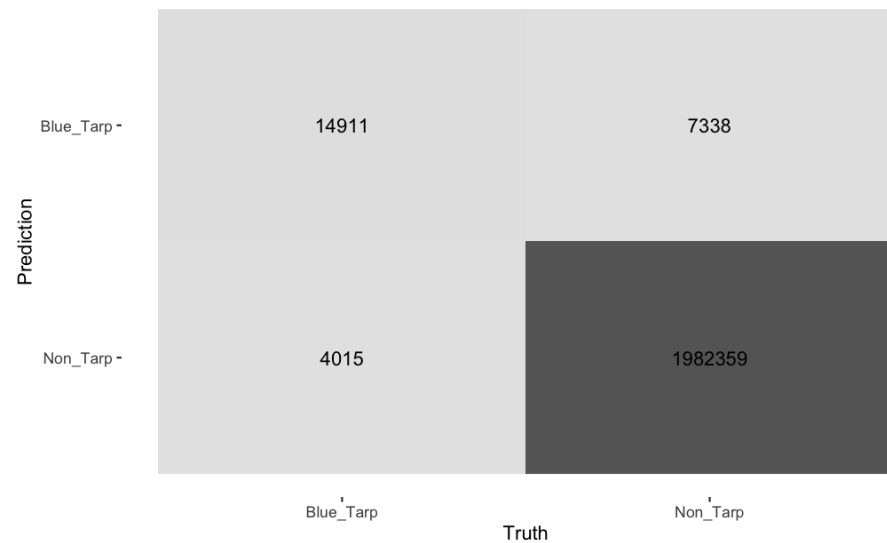
Random Forest on Training and Test Sets

| Metric   | Train Estimate | Test Estimate |
|----------|----------------|---------------|
| accuracy | 0.9968         | 0.9946        |
| f_meas   | 0.9500         | 0.7372        |
| recall   | 0.9442         | 0.7977        |
| roc_auc  | 0.9997         | 0.9806        |

# Random Forest

## Comparison

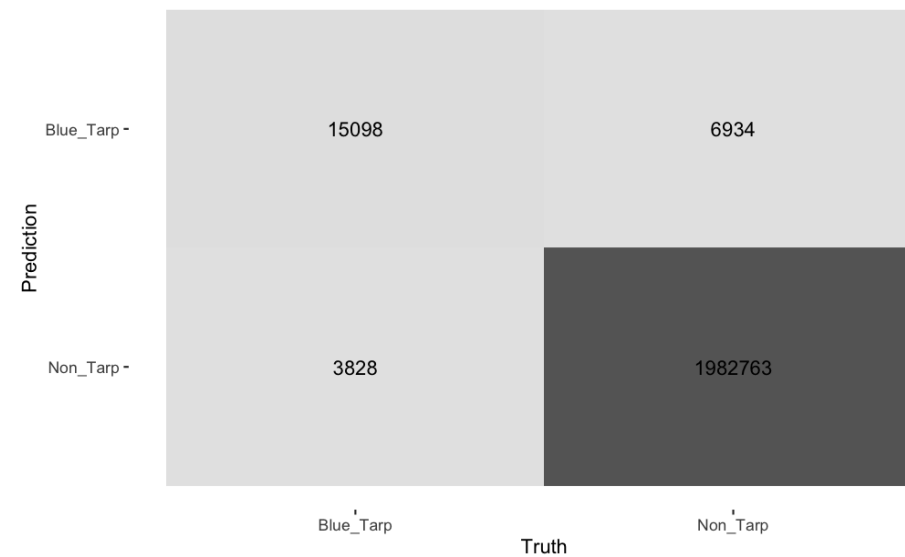
### First Model



Random Forest on Training and Test Sets

| Metric   | Train Estimate | Test Estimate |
|----------|----------------|---------------|
| accuracy | 0.9970         | 0.9943        |
| f_meas   | 0.9535         | 0.7243        |
| recall   | 0.9475         | 0.7879        |
| roc_auc  | 0.9997         | 0.9804        |

### Second Model



Random Forest on Training and Test Sets

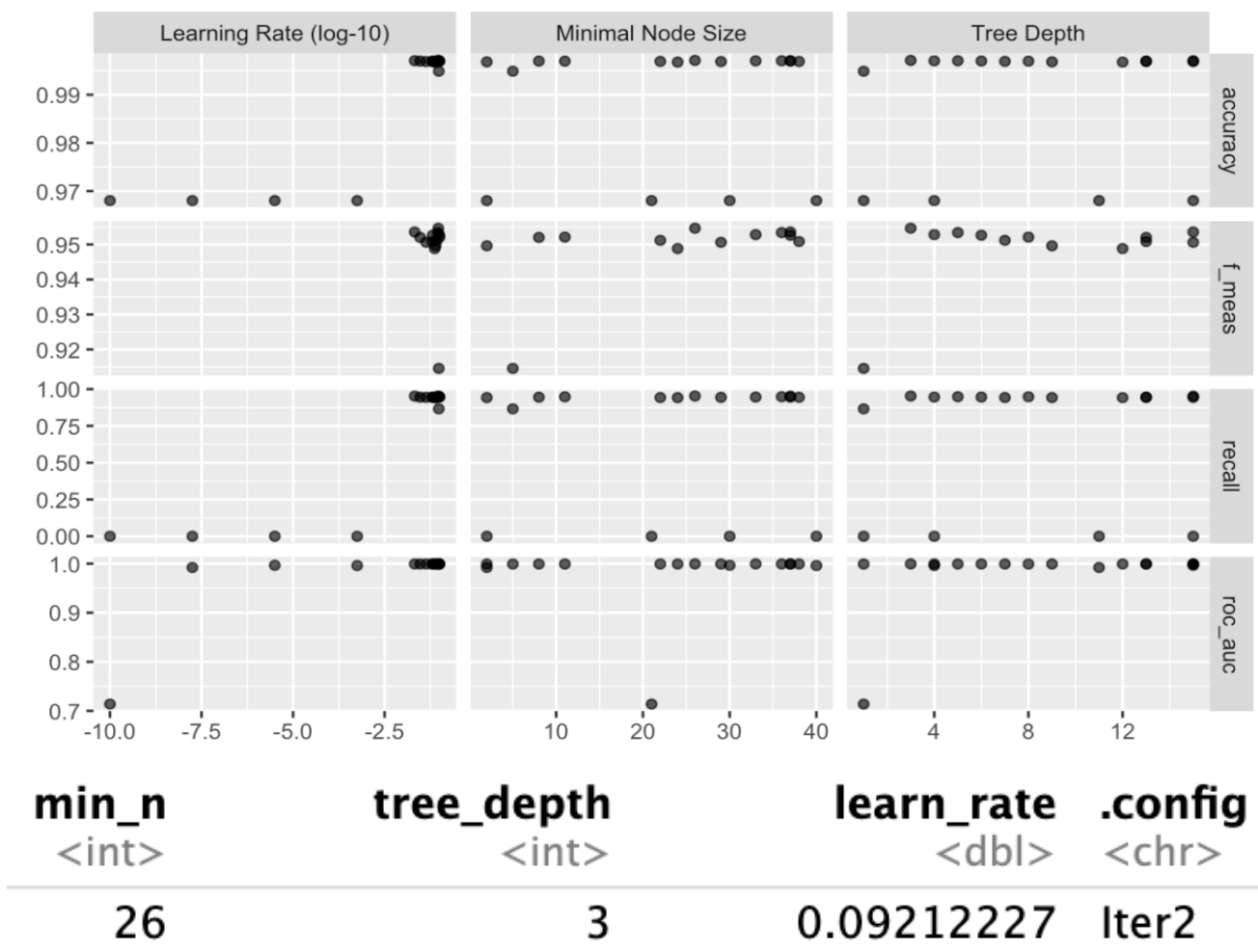
| Metric   | Train Estimate | Test Estimate |
|----------|----------------|---------------|
| accuracy | 0.9968         | 0.9946        |
| f_meas   | 0.9500         | 0.7372        |
| recall   | 0.9442         | 0.7977        |
| roc_auc  | 0.9997         | 0.9806        |



# Boosting

## Model Tuning

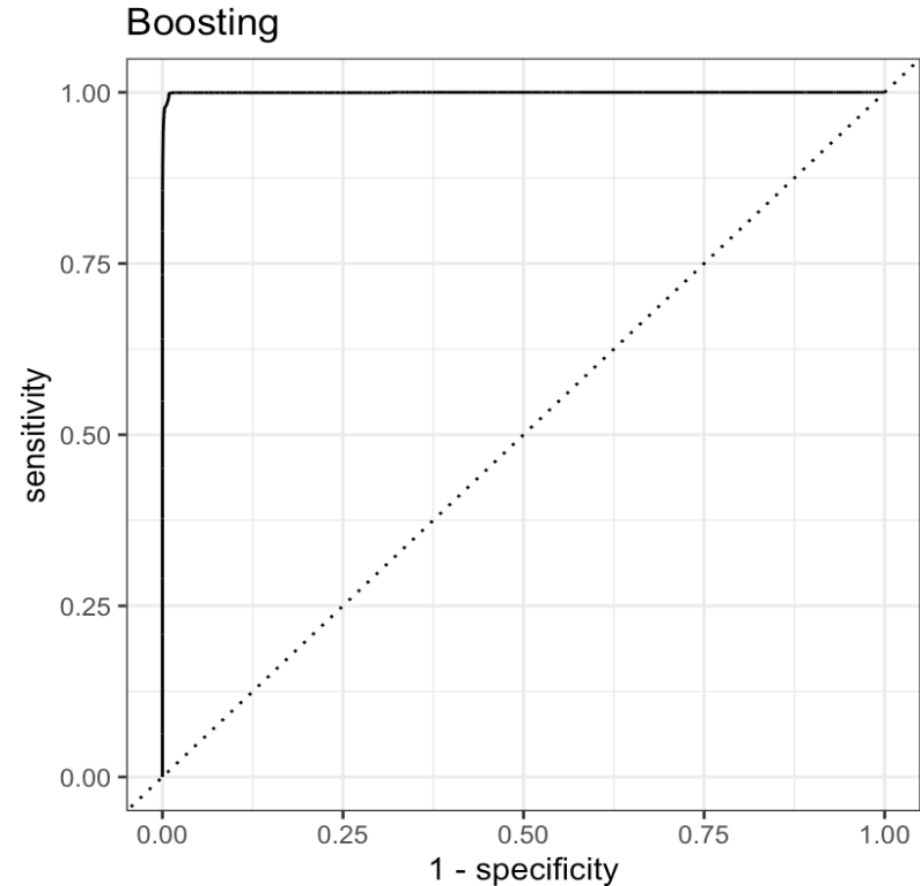
- The hyperparameters that we decided to tune were:
  - Tree Depth (tree\_depth)
  - Learning Rate (learn\_rate)
  - Minimal Node Size (min\_n)
- The hyperparameters were tuned to maximize ROC AUC.
  - Min\_n = 26
  - Tree\_depth = 3
  - Learn\_Rate = .0921
  - Number of Trees = 500 (fixed)



# Boosting

## Model Evaluation

- The ROC curve shows how well the model distinguishes between "Blue Tarp" and "Not Blue Tarp".
- The curve hugs the top-left corner, which could mean that the model performs well at distinguishing between the two levels.



# Boosting

## Model Evaluation

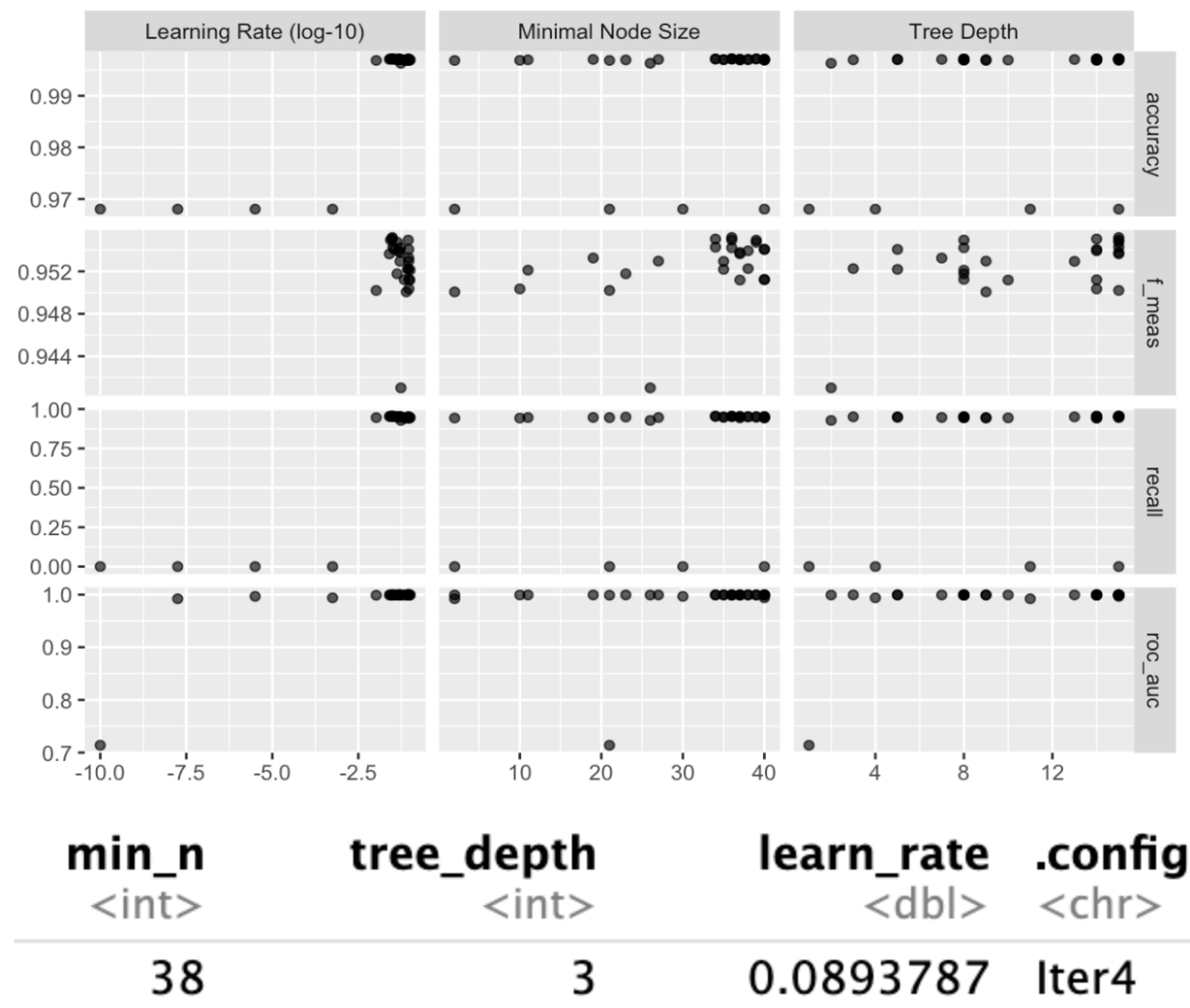
- When assessing the model on the test data, the accuracy and ROC AUC values remain high.
- However, both the recall and f\_measure values drop from the training to the test dataset.
- I can try to improve the model by decreasing the number of trees

| <b>.metric</b><br><chr> | <b>train</b><br><dbl> | <b>test</b><br><dbl> |
|-------------------------|-----------------------|----------------------|
| accuracy                | 0.9971221             | 0.9927023            |
| f_meas                  | 0.9546602             | 0.6360416            |
| recall                  | 0.9527911             | 0.7501381            |
| roc_auc                 | 0.9995579             | 0.9777980            |

# Boosting

## Model Evaluation

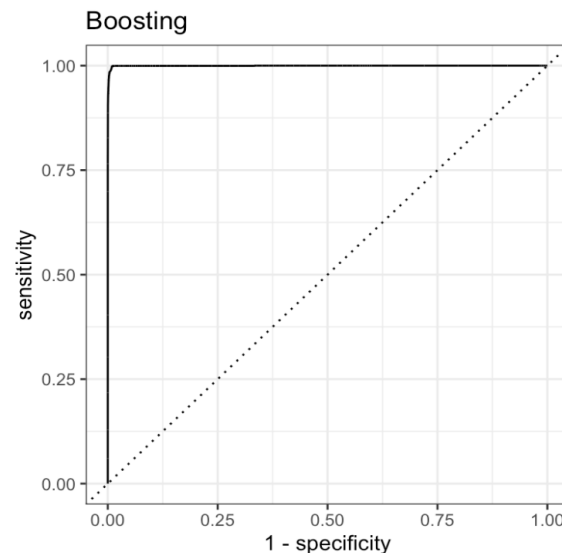
- The hyperparameters that were tuned to maximize ROC AUC.
  - Min\_n = 38
  - Tree\_depth = 3
  - Learn\_Rate = .08938
  - Number of Trees = 250
- After selecting the best hyperparameters, we finalized the workflow and fit the model on the entire training dataset.



# Boosting

## Model Evaluation

- The ROC curve still looks very good even after pruning which suggests the model is effective.
- When assessing the pruned model on the test data, the accuracy and ROC AUC values remain high. The F measure and recall slightly improve in the pruned model.
- Overall, the pruned boosted model performs better.
- According to the confusion matrix, the error rate of the model is .0067.



| Prediction    | Truth         |           |
|---------------|---------------|-----------|
|               | Not Blue Tarp | Blue Tarp |
| Not Blue Tarp | 1680629       | 3122      |
| Blue Tarp     | 8296          | 11358     |

| .metric<br><chr> | train<br><dbl> | test<br><dbl> |
|------------------|----------------|---------------|
| accuracy         | 0.9969640      | 0.9932970     |
| f_meas           | 0.9522614      | 0.6654948     |
| recall           | 0.9508680      | 0.7843923     |
| roc_auc          | 0.9995680      | 0.9757225     |

# Support Vector Machines

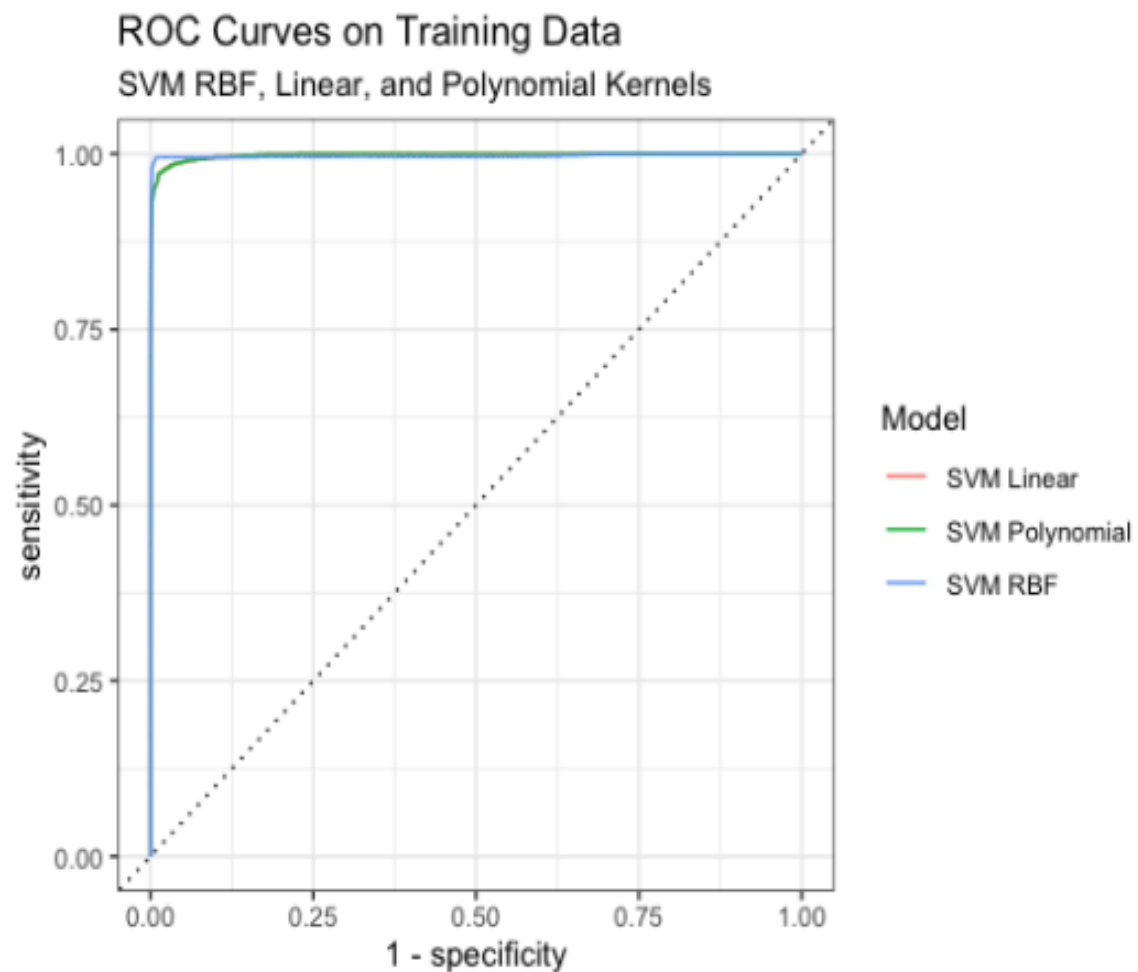
## Methodology

---

1. Used 3 model types, all trained with the kernlab engine:
  - SVM with RBF kernel: tuned over cost and rbf\_sigma
  - SVM with Linear kernel: tuned over cost
  - SVM with Polynomial kernel: tuned over cost, degree, and scale\_factor
2. For tuning and training, we ran a 10-fold stratified cross-validation on the training set. Best parameters selected using ROC-AUC.
3. Metrics were computed for both training and test sets using ROC curves and PCA projections that highlight correct vs. Incorrect classifications

# Support Vector Machines

## Results

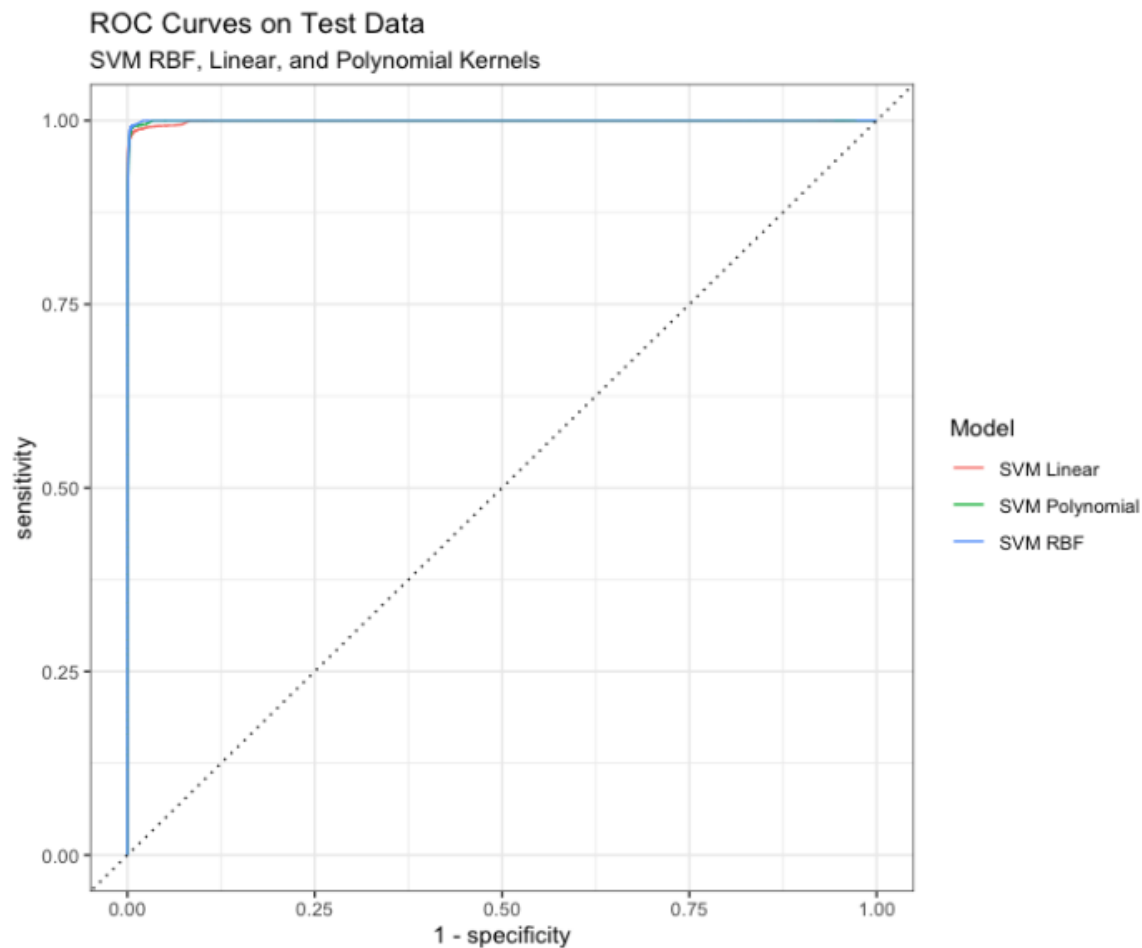


SVM Training Performance Metrics

| model          | accuracy | sens   | spec   | precision | recall | f_meas | roc_auc |
|----------------|----------|--------|--------|-----------|--------|--------|---------|
| SVM RBF        | 0.9971   | 0.9416 | 0.9989 | 0.9665    | 0.9416 | 0.9539 | 0.9968  |
| SVM Linear     | 0.9954   | 0.8892 | 0.9989 | 0.9651    | 0.8892 | 0.9256 | 0.9978  |
| SVM Polynomial | 0.9954   | 0.8882 | 0.9989 | 0.9651    | 0.8882 | 0.9251 | 0.9978  |

# Support Vector Machines

## Results



SVM Test Set Performance

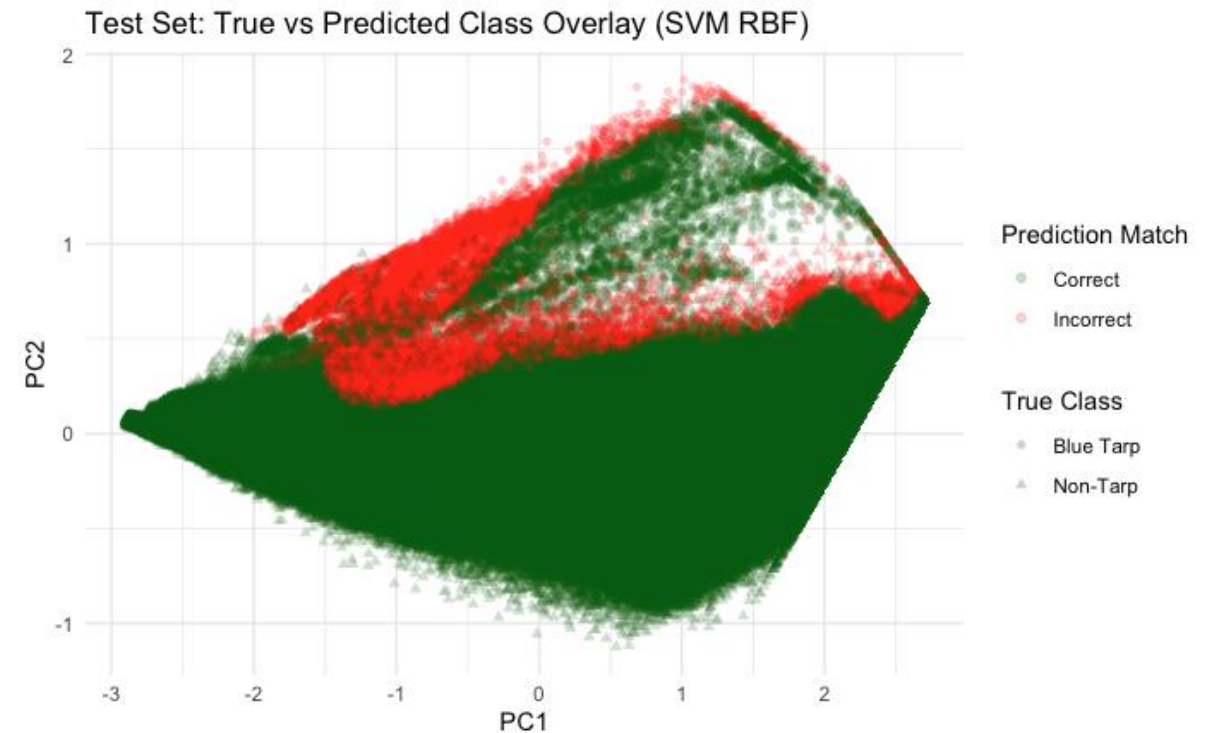
| model          | accuracy | sensitivity | specificity | precision | recall | f_meas | roc_auc |
|----------------|----------|-------------|-------------|-----------|--------|--------|---------|
| SVM RBF        | 0.9944   | 0.9916      | 0.9945      | 0.5658    | 0.9916 | 0.7205 | 0.9997  |
| SVM Linear     | 0.9829   | 0.9883      | 0.9829      | 0.2957    | 0.9883 | 0.4552 | 0.9991  |
| SVM Polynomial | 0.9941   | 0.9906      | 0.9941      | 0.5495    | 0.9906 | 0.7068 | 0.9996  |



# Support Vector Machines

## Results

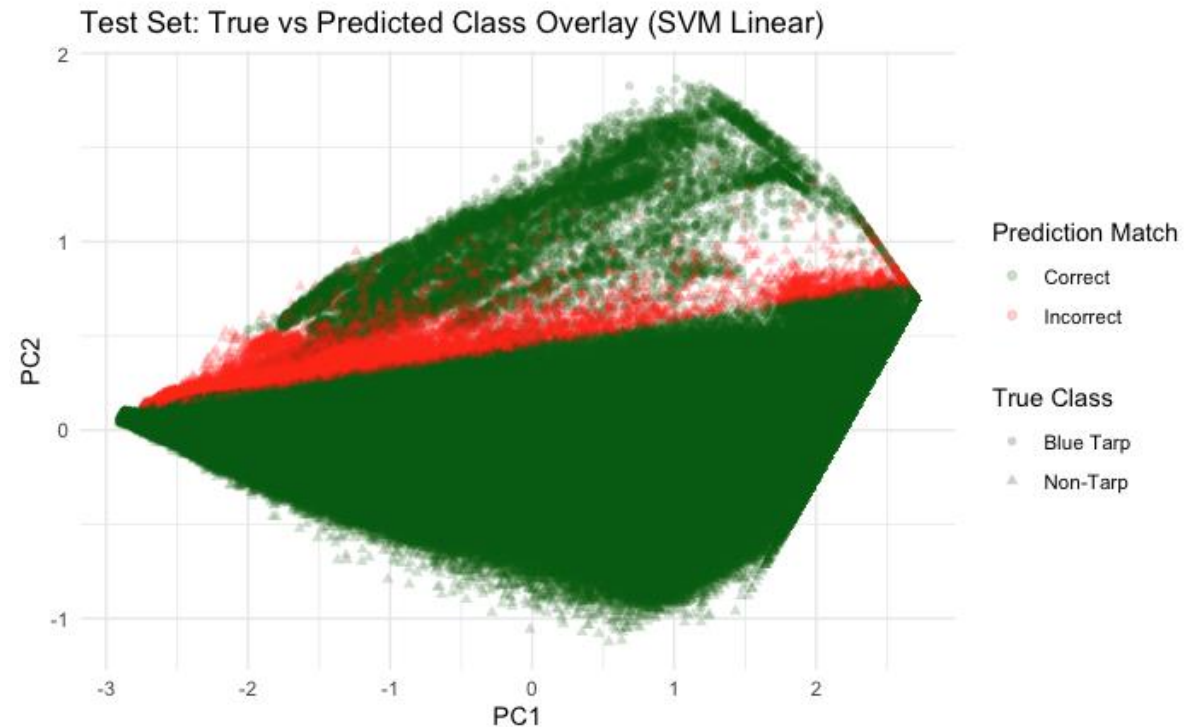
- The PCA plots reduces the 3D RGB feature space into two dimensions: PC1 and PC2.
- Points are colored by prediction correctness (Correct vs Incorrect) and shaped by the true class (Blue Tarp vs Non-Tarp)



# Support Vector Machines

## Results

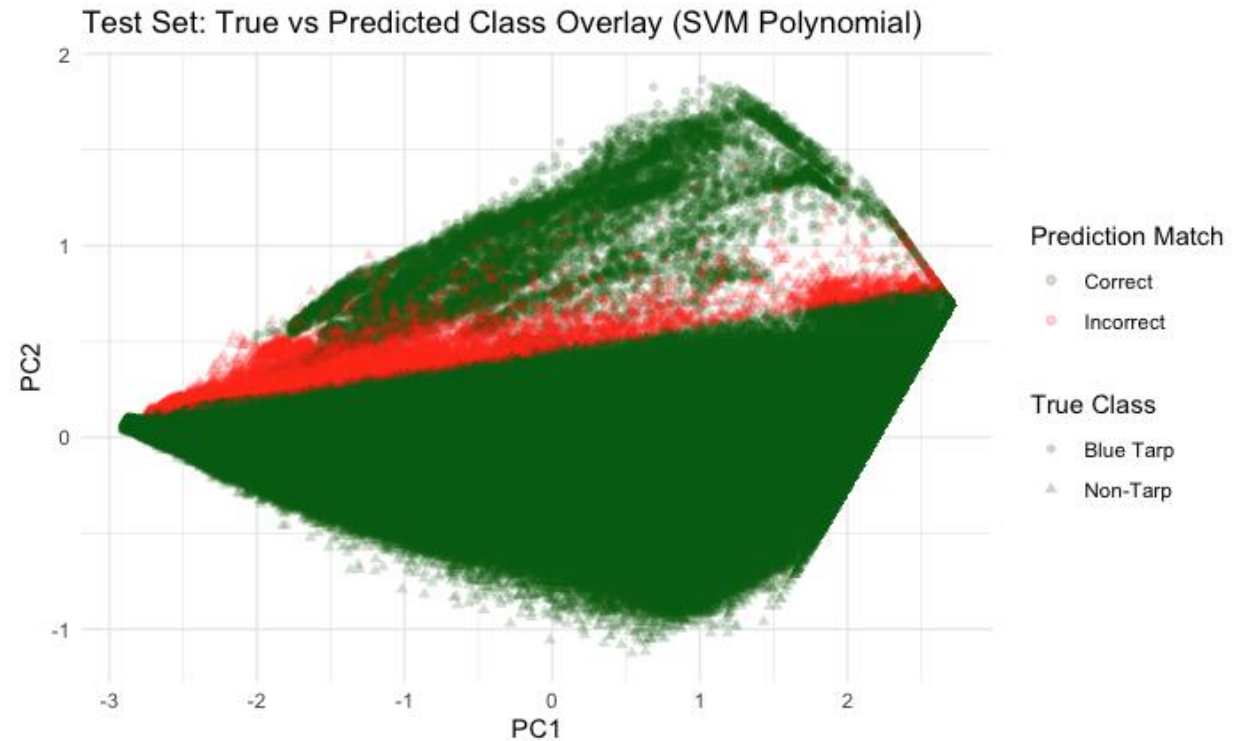
- The PCA plots reduces the 3D RGB feature space into two dimensions: PC1 and PC2.
- Points are colored by prediction correctness (Correct vs Incorrect) and shaped by the true class (Blue Tarp vs Non-Tarp)



# Support Vector Machines

## Results

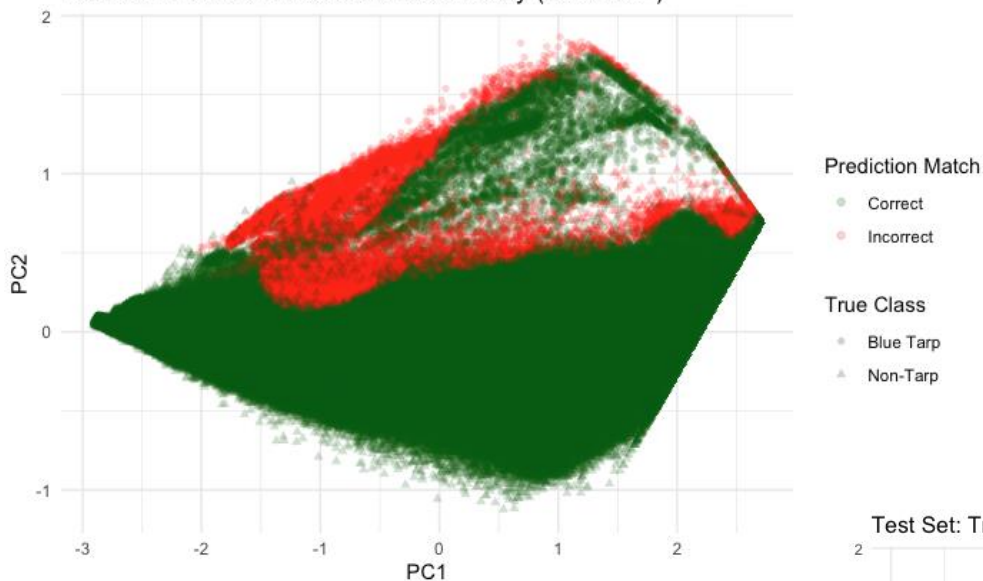
- The PCA plots reduces the 3D RGB feature space into two dimensions: PC1 and PC2.
- Points are colored by prediction correctness (Correct vs Incorrect) and shaped by the true class (Blue Tarp vs Non-Tarp)



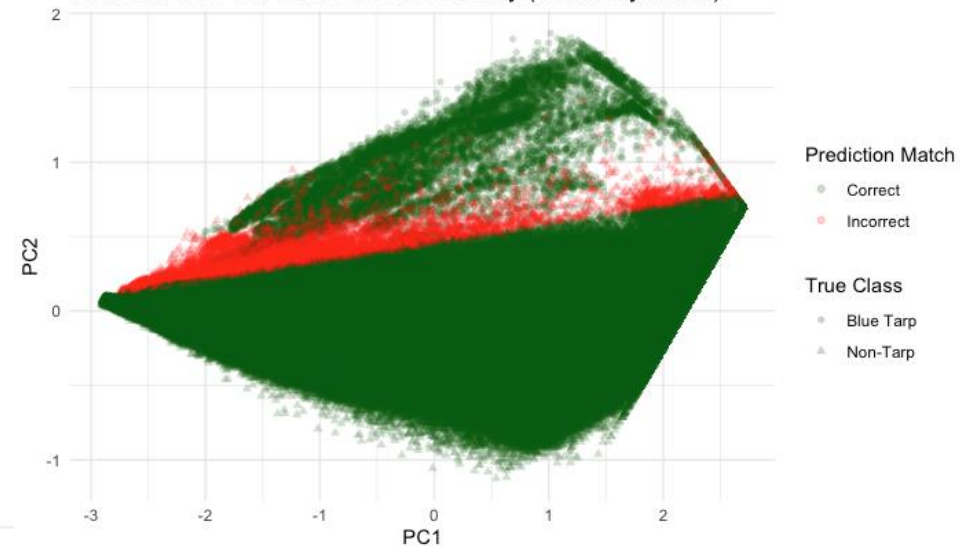
# Support Vector Machines

## Results

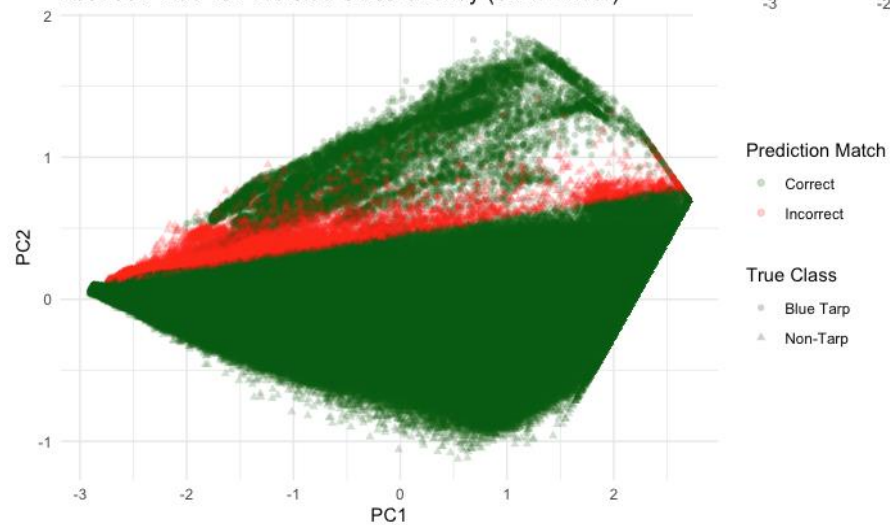
Test Set: True vs Predicted Class Overlay (SVM RBF)



Test Set: True vs Predicted Class Overlay (SVM Polynomial)



Test Set: True vs Predicted Class Overlay (SVM Linear)



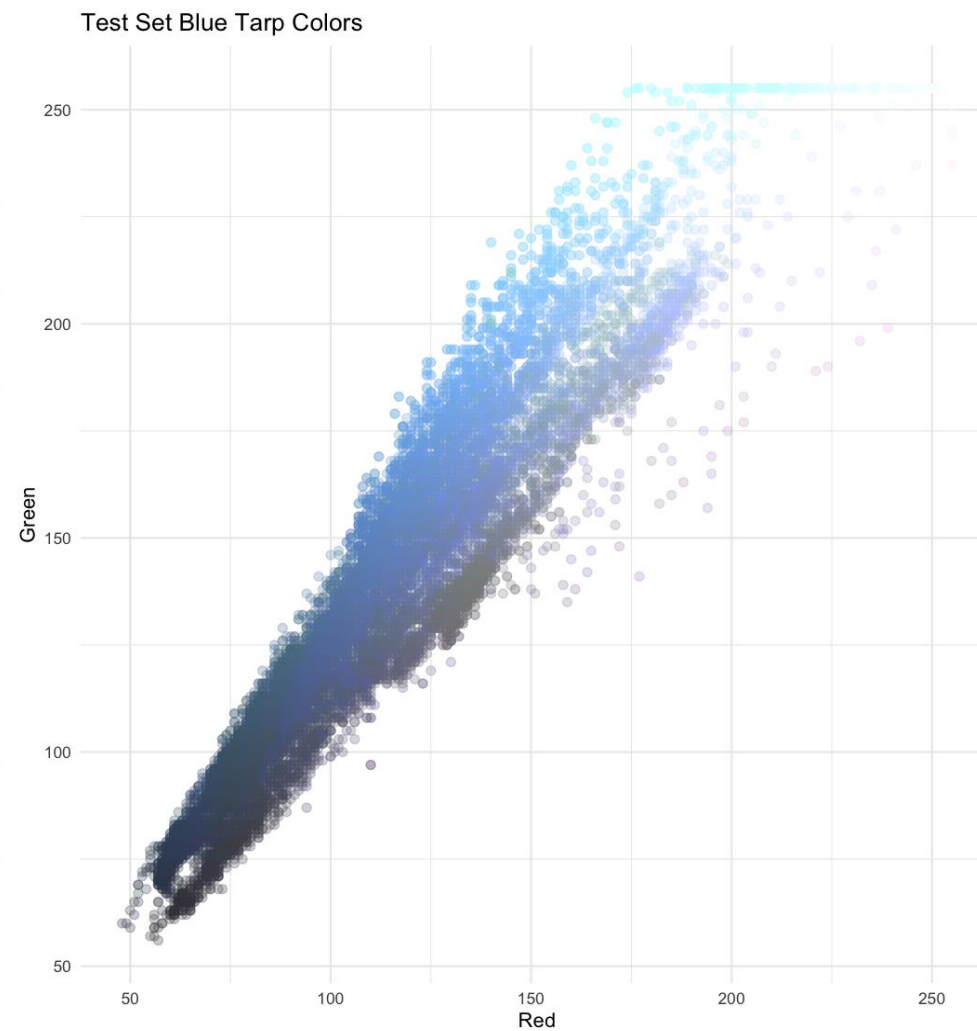
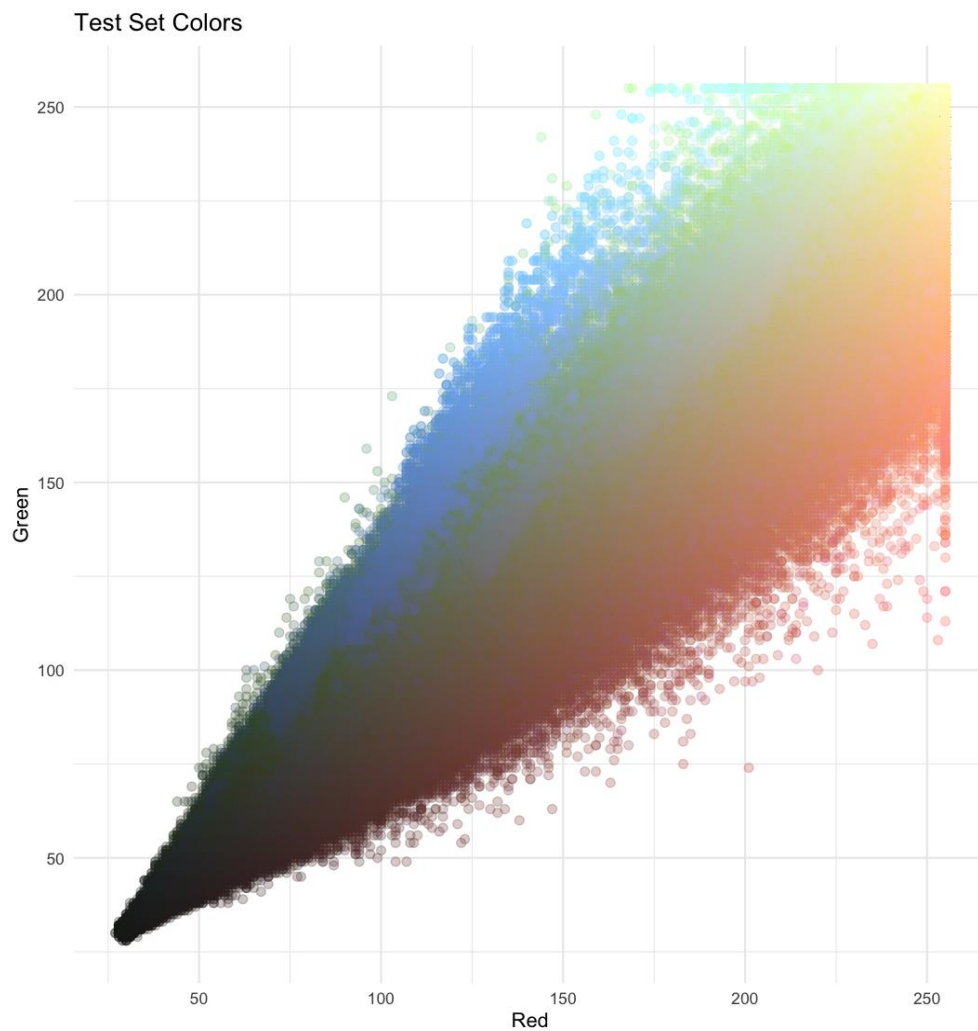
# Comparison of Models

Performance Metrics

|                     | Accuracy | Recall | F Measure | ROC AUC |
|---------------------|----------|--------|-----------|---------|
| LDA                 | 0.976    | 0.839  | 0.988     | 0.992   |
| Logistic Regression | 0.904    | 0.988  | 0.949     | 0.999   |
| QDA                 | .977     | 0.695  | 0.988     | 0.992   |
| KNN                 | 0.997    | 0.965  | 0.929     | 0.993   |
| Random Forest       | 0.995    | 0.798  | 0.737     | 0.981   |
| Boosted Forest      | 0.993    | 0.784  | 0.665     | 0.976   |
| SVM RBF             | 0.994    | 0.992  | 0.721     | 0.999   |
| SVM Linear          | 0.983    | 0.988  | 0.452     | 0.999   |
| SVM Polynomial      | 0.994    | 0.994  | 0.701     | 0.999   |

# Comparison of Models

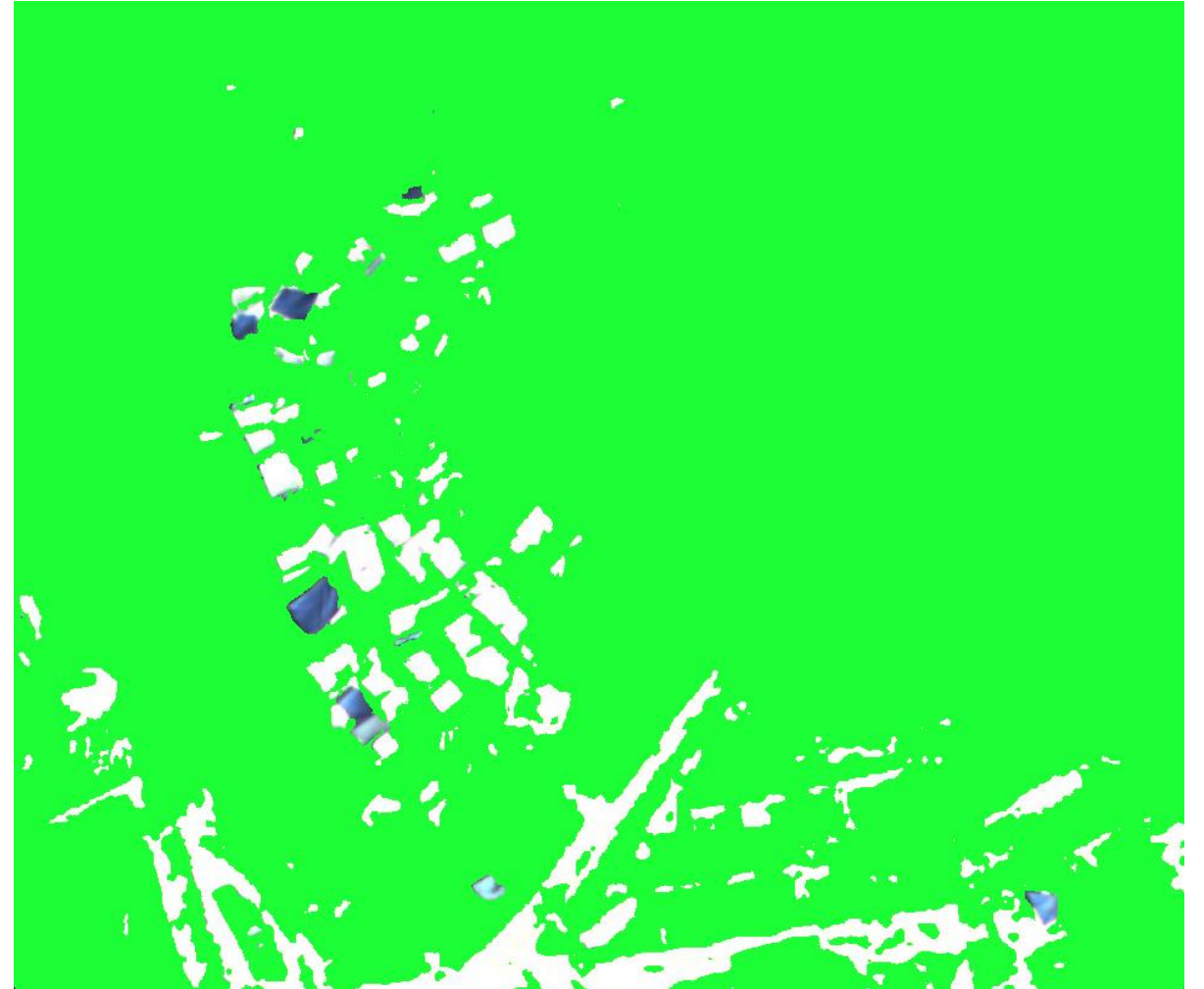
Color Values





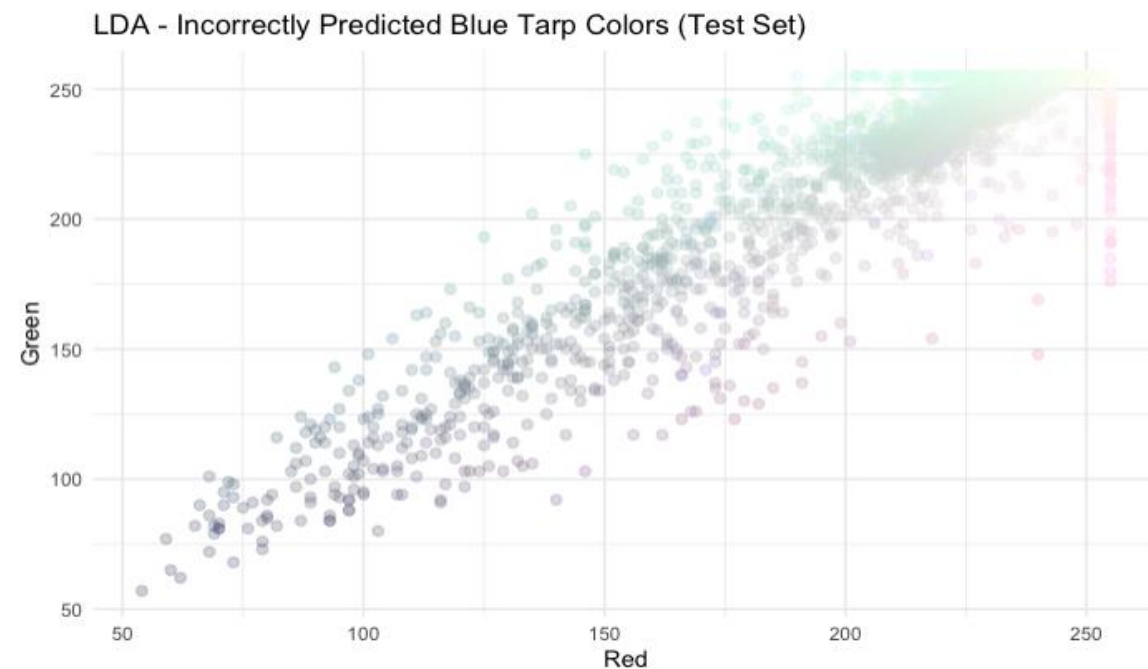
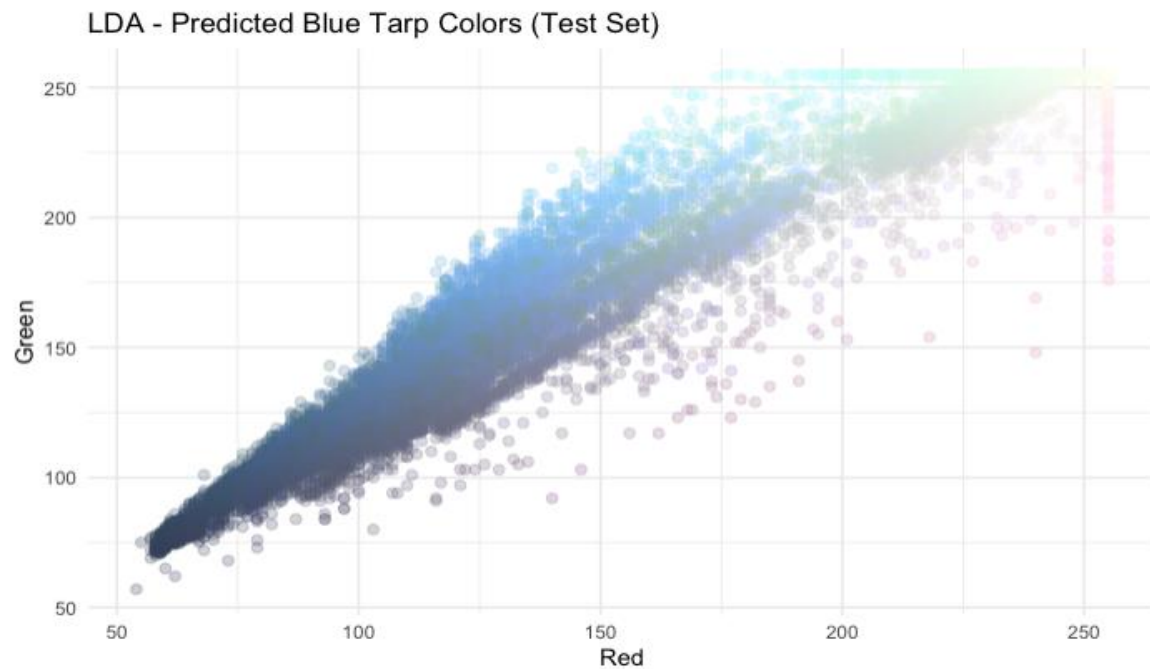
# Comparison of Models

Maps - LDA



# Comparison of Models

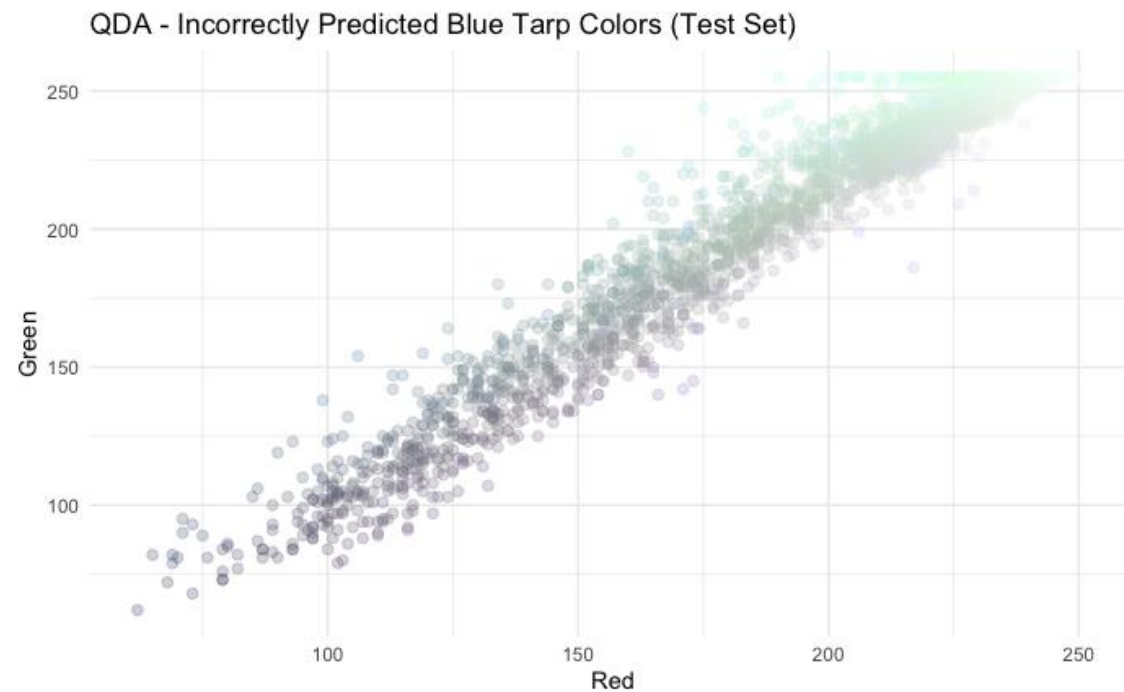
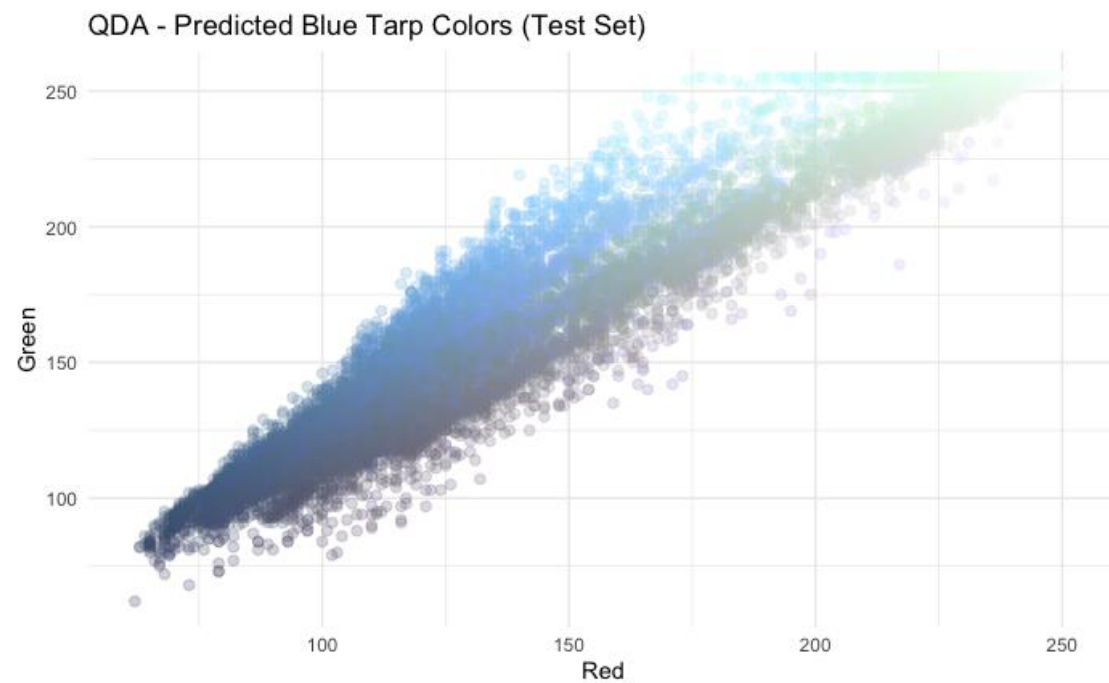
Colors - LDA





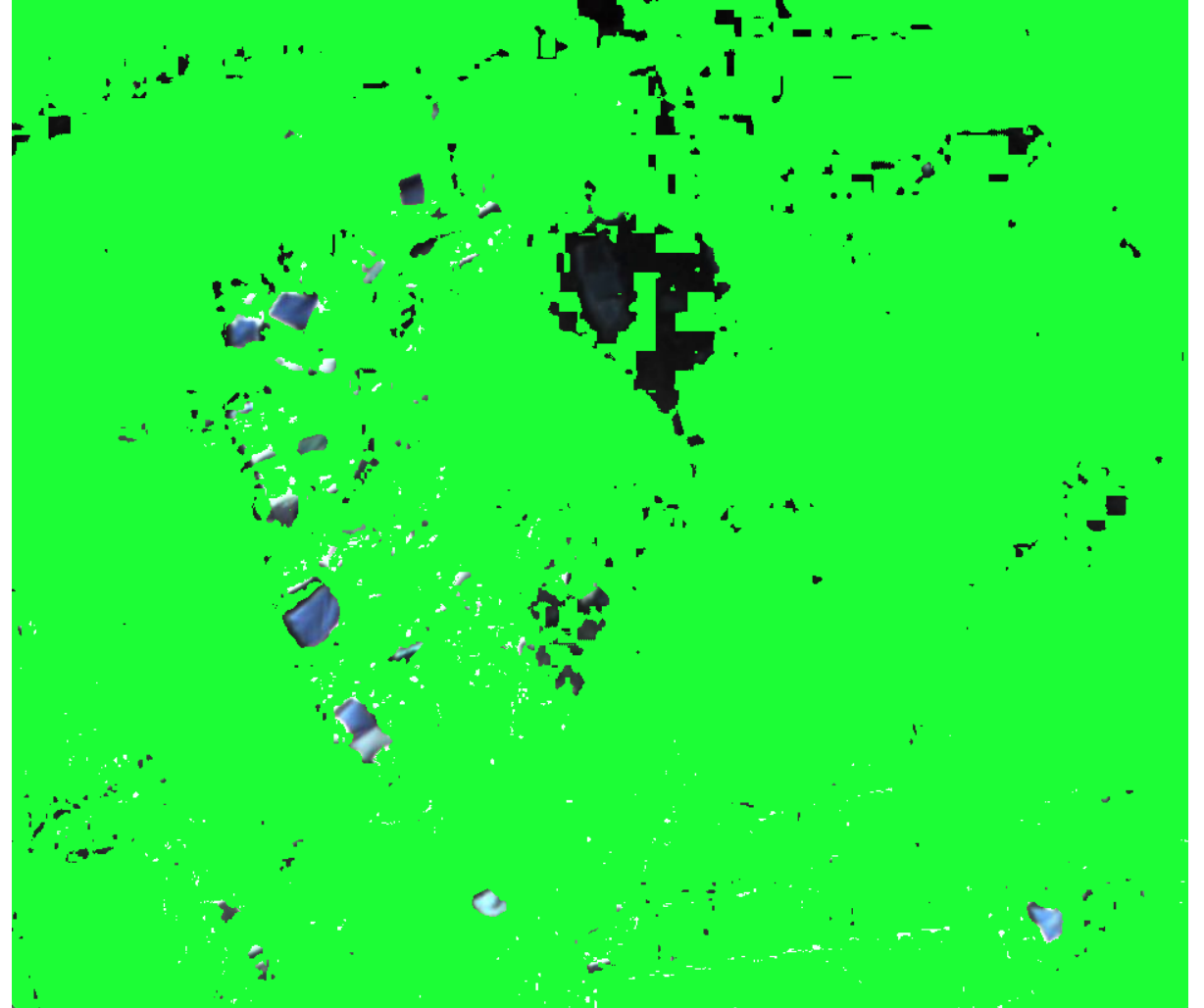
# Comparison of Models

Colors - QDA



# Comparison of Models

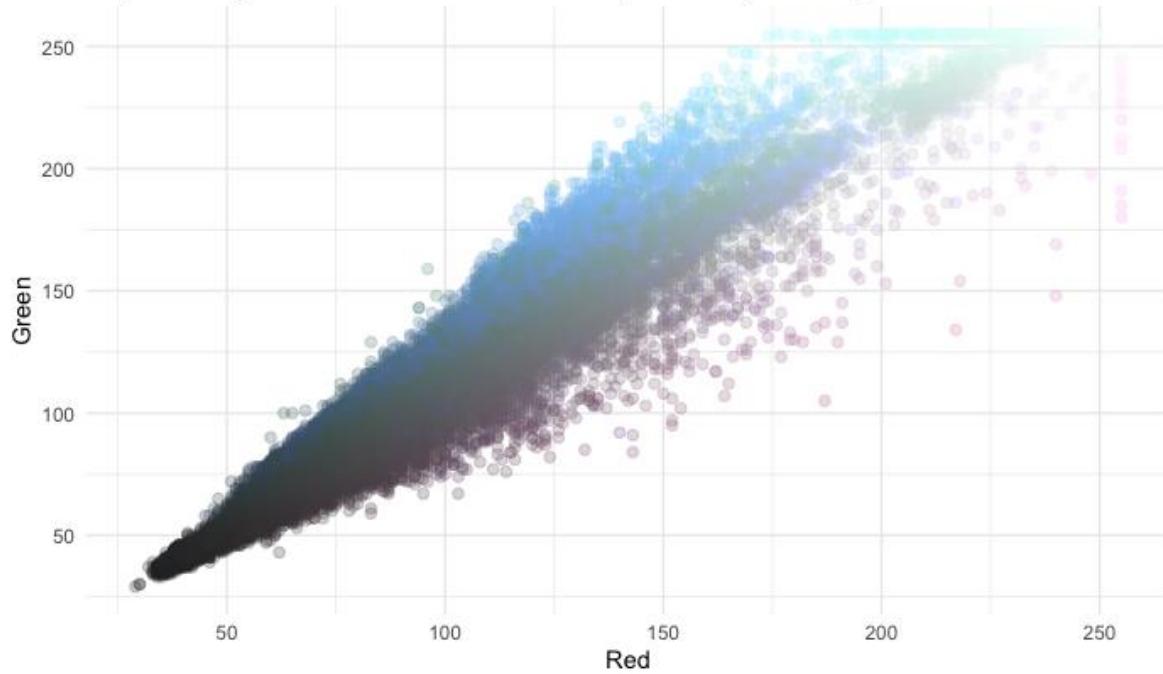
Maps – Logistic Regression



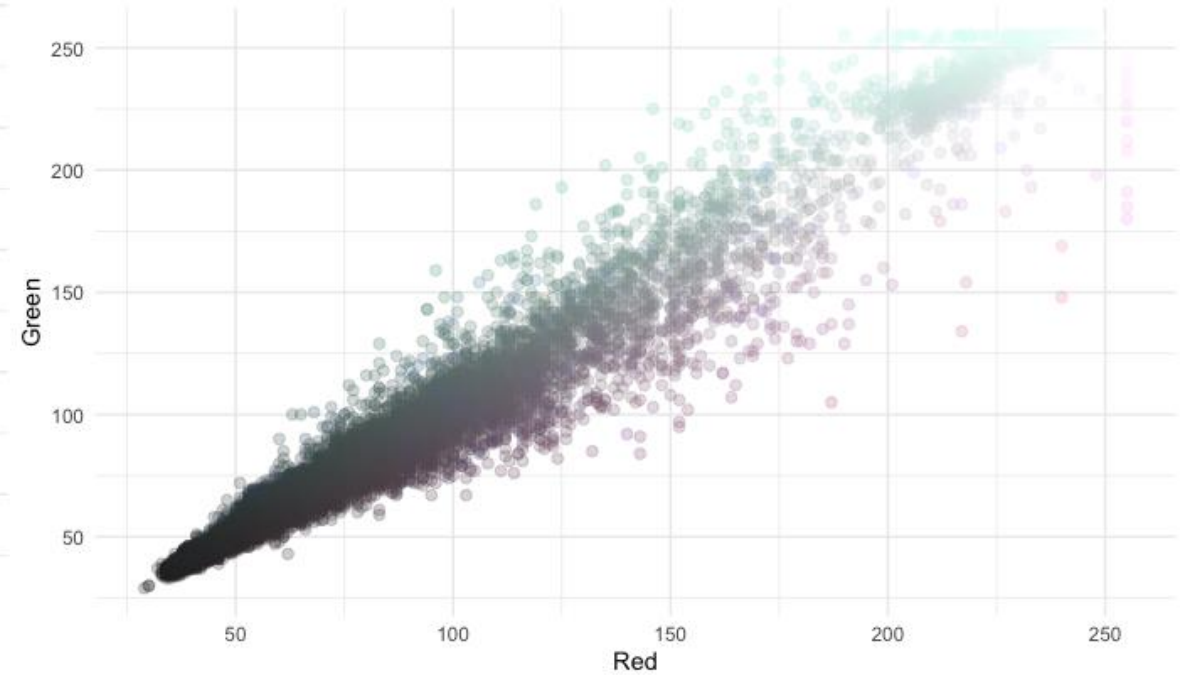
# Comparison of Models

Colors - Logistic Regression

Logistic Regression - Predicted Blue Tarp Colors (Test Set)



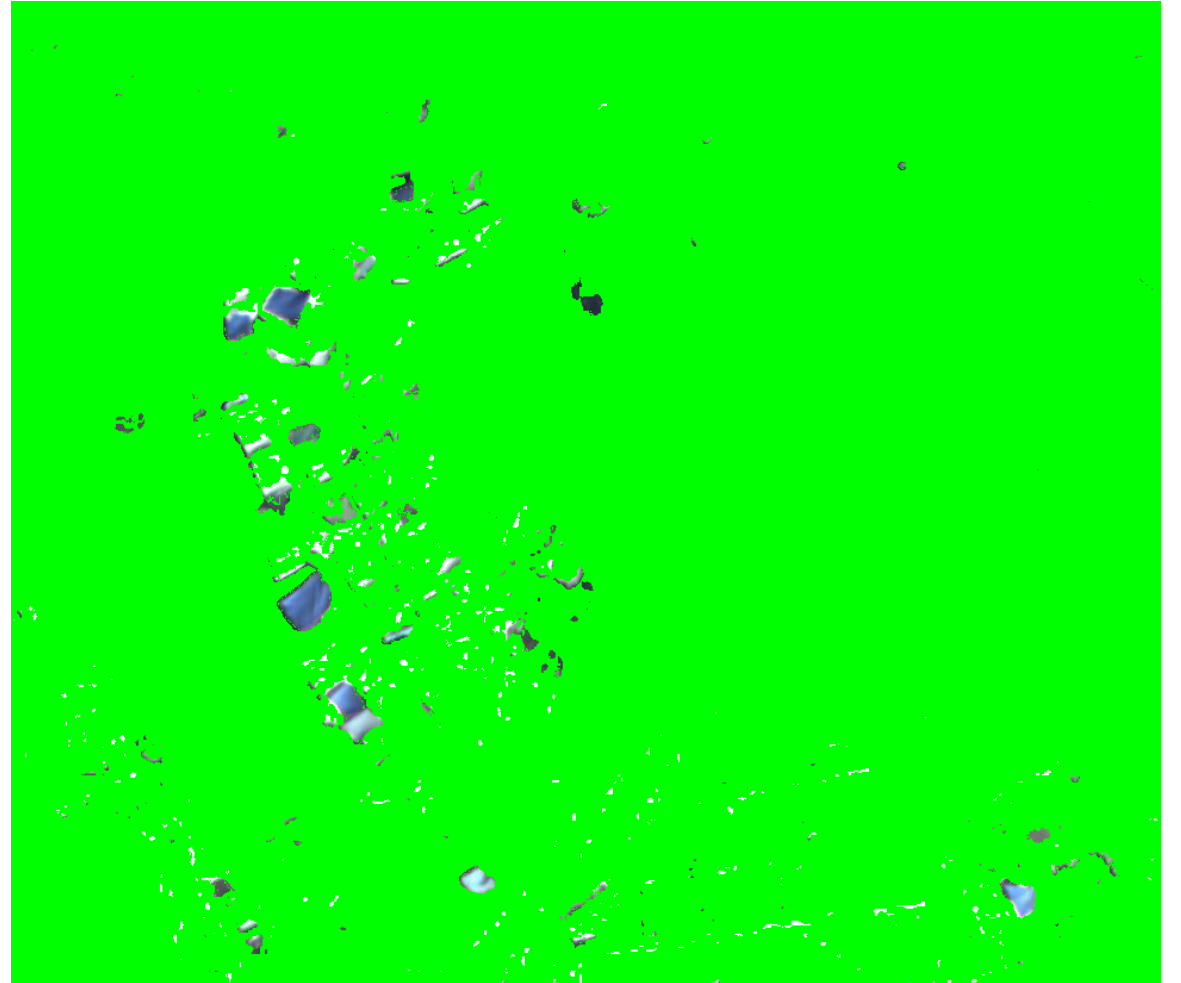
Logistic Regression - Incorrectly Predicted Blue Tarp Colors (Test Set)





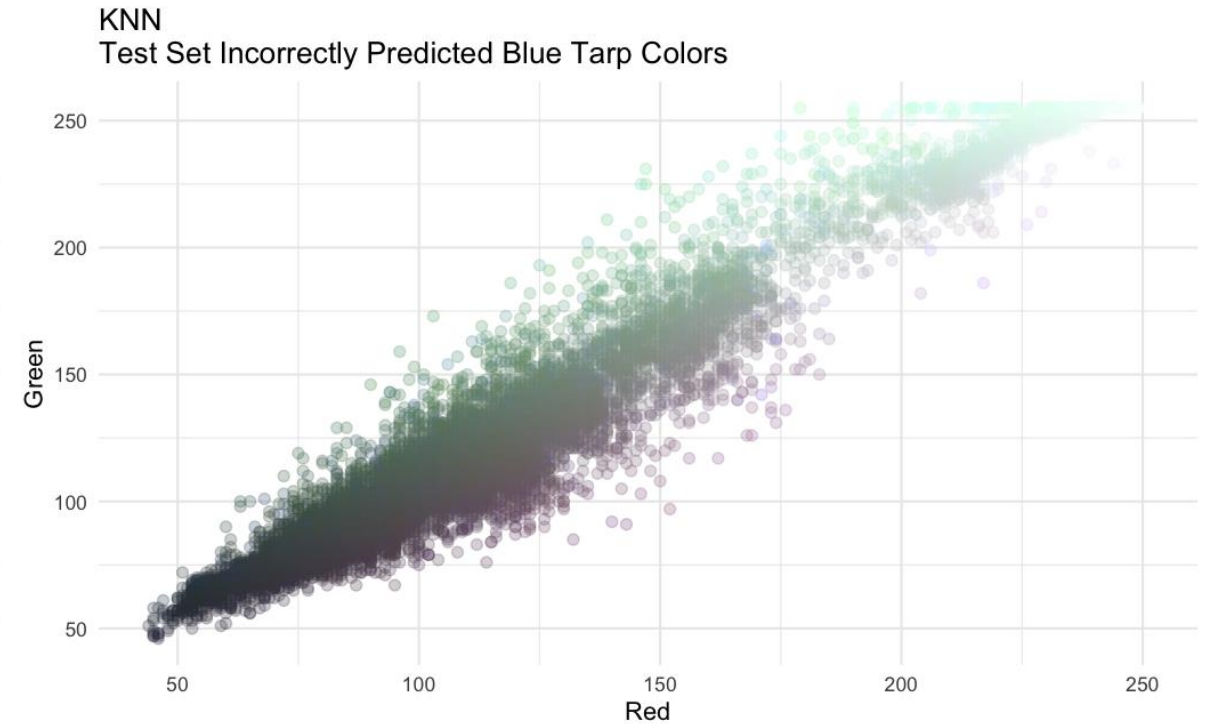
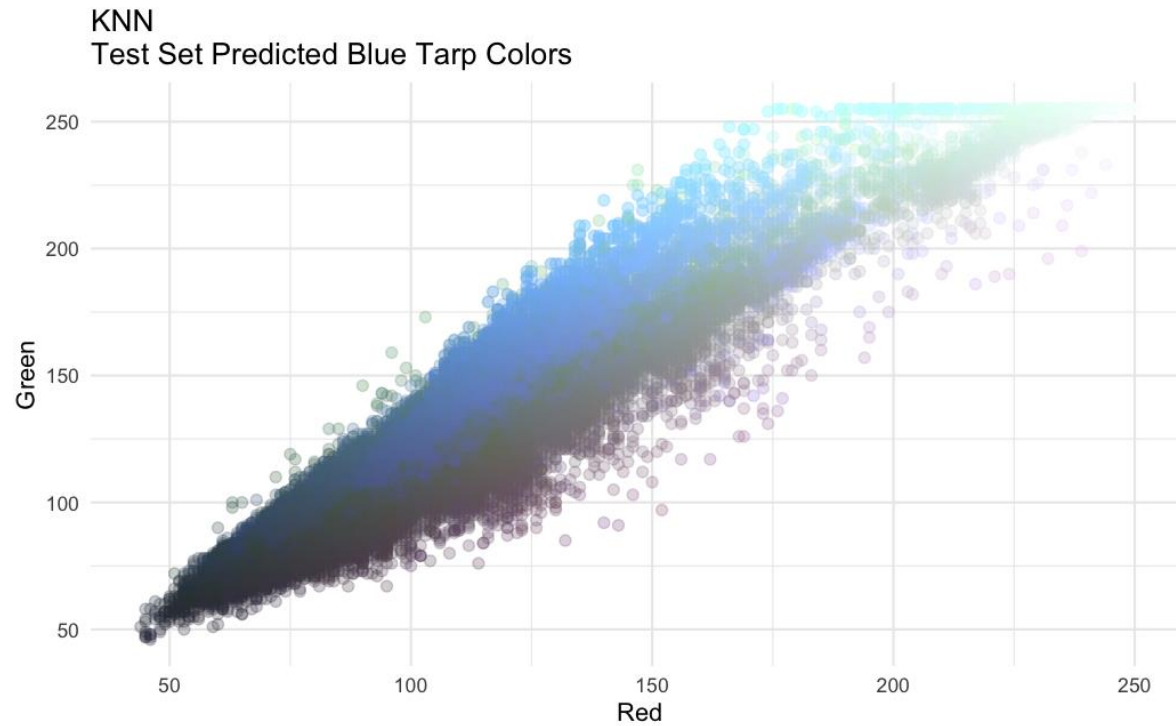
# Comparison of Models

Maps - KNN



# Comparison of Models

Colors - KNN





# Conclusion

## K-Nearest Neighbors

---

- K-Nearest Neighbors is the best model.
- Among the best F1 with a very high recall, highest accuracy.
- Value of recall in life/death scenarios- we want to identify everyone.
- Passes the eye test.